Een Open-Source Benadering voor Daksch detectie met Drones: Proof of Concept voor Kleine Zelfstandige Dakwerkers.

Bachelorproef.

Bas Stokmans.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. S. De Gheselle
Co-promotor: Dhr. P. Zeeuws
Academiejaar: 2024–2025
Eerste examenperiode

Departement IT en Digitale Innovatie .

HO
GENT

Woord vooraf

De keuze voor dit onderzoeksonderwerp kwam voort uit mijn fascinatie voor kunstmatige intelligentie en de praktische toepassingen ervan. Ik heb eerder met YOLO-objectdetectie geprobeerd te werken en deze bachelorproef gaf me de unieke kans om objectdetectie echt grondig te begrijpen en toe te passen op een maatschappelijk probleem.

Het onderwerp van dakschade-detectie met drones sprak me aan vanwege de efficiëntie en de veiligheid die deze technologie kan bieden. De combinatie van drone-technologie en real-time AI-detectie maakt het mogelijk om inspectieprocessen niet alleen veiliger te maken, maar ook toegankelijker voor kleinere bedrijven. Dit past perfect bij mijn overtuiging dat technologie ontwikkeld moet worden om praktische problemen op te lossen.

Deze bachelorproef was voor mij ook een persoonlijke uitdaging. Mijn afstudeerrichting is Mobile & Web Development, maar dit onderzoek ligt volledig in de AI-afstudeerrichting. Ik ben er trots op dat ik zoveel nieuwe kennis heb kunnen verwerven en een werkend proof of concept heb kunnen ontwikkelen dat een goede basis vormt om verder op uit te breiden.

Verschillende mensen hebben een cruciale rol gespeeld in de totstandkoming van dit werk. Vooreerst wil ik mijn promotor, dhr. S. De Gheselle, bedanken voor zijn begeleiding en vertrouwen in dit project. Een bijzondere dank gaat uit naar mijn co-promotor, dhr. P. Zeeuws van Stad Mechelen, die niet alleen zijn expertise deelde maar ook de praktijktests mogelijk maakte. Dankzij hem kon ik het systeem testen op echte gebouwen in Mechelen, waaronder de Stadsschouwburg en Lamot.

Ook mijn collega's tijdens mijn stage hebben bijgedragen met hun inzichten en feedback, wat me heeft geholpen om het onderzoek vanuit verschillende perspectieven te benaderen.

Tot slot wil ik mijn familie en vrienden bedanken voor hun geduld en steun tijdens de intensieve periode van onderzoek en ontwikkeling.

Met deze bachelorproef hoop ik mijn steentje te hebben bijgedragen aan de ontwikkeling van toegankelijke AI-toepassingen voor de bouwsector. Ik hoop dat dit werk een basis kan vormen voor verdere innovatie op dit gebied.

Samenvatting

Traditionele dakinspecties in de bouwsector zijn gevaarlijk, tijdrovend en arbeidsintensief. Valpartijen vanaf daken zijn verantwoordelijk voor meer dan één op drie dodelijke ongevallen in de bouwsector. Hoewel drone-technologie een veilig alternatief biedt, blijft het handmatig analyseren van verzamelde beelden inefficiënt en vereist het vaak meerdere vluchten wanneer verdachte gebieden nader onderzocht moeten worden.

Het centrale probleem dat dit onderzoek aanpakt is de integratie van real-time beeldverwerking en schadedetectie in het drone-inspectieproces, zodat verdachte gebieden onmiddellijk kunnen worden geïdentificeerd tijdens dezelfde vlucht.

De hoofdonderzoeksvraag luidt: “Hoe kan een real-time dakschade-detectiesysteem worden ontwikkeld dat gebruikmaakt van drone-beelden en YOLO-objectdetectie op een draagbaar computersysteem met optimale balans tussen verwerkingssnelheid en detectienauwkeurigheid?”

Het onderzoek omvatte een vergelijkende analyse van verschillende YOLO-varianten (YOLOv5 en YOLOv8) en de implementatie van SQRT-balancing voor ongebalanceerde datasets. Een testopstelling werd ontwikkeld waarbij dakschadetypen onder gecontroleerde omstandigheden werden vastgelegd en gedetecteerd via RTMP-streaming naar een laptop voor real-time verwerking.

De belangrijkste resultaten bewijzen dat real-time dakschade-detectie technisch haalbaar is. YOLOv8 presteerde beter dan YOLOv5, en SQRT-balancing verbeterde de detectie van ondervertegenwoordigde schadetypen. Het systeem behaalde real-time verwerking met 40-45 FPS op GPU-hardware en een totale latentie van 520-540 milliseconden. Detectieresultaten varieerden afhankelijk van de testomgeving, met betere prestaties op vergelijkbare daktypen als de trainingsdata en lagere scores op materiaal dat niet in de dataset voorkwam.

Deze bevindingen tonen aan dat het concept werkt en een solide basis vormt voor verdere ontwikkeling. Het prototype demonstreert de haalbaarheid van real-time dakschade-detectie met consumer hardware en identificeert concrete verbeterpunten voor toekomstige implementaties, zoals datasetuitbreiding en optimalisatie voor verschillende daktypen.

Inhoudsopgave

Lijst van figuren ix

Lijst van tabellen xi

Lijst van codefragmenten xii

1 Inleiding 1
- 1.1 Probleemstelling . . . 1
- 1.2 Onderzoeksvraag . . . 2
- 1.3 Onderzoeksdoelstelling . . . 2
- 1.4 Opzet van deze bachelorproef . . . 3

2 Stand van zaken 5
- 2.1 Drone-Technologie in de Bouwsector . . . 5
  - 2.1.1 Voordelen van Drone-Inspecties . . . 5
  - 2.1.2 Nadelen en Uitdagingen . . . 6
- 2.2 Objectdetectie en Computer Vision voor Schadeanalyse . . . 6
  - 2.2.1 Two-Stage vs One-Stage Detectors . . . 7
- 2.3 YOLO: Een Revolutionaire Aanpak in Objectdetectie . . . 7
  - 2.3.1 Werkingsprincipe van YOLO . . . 7
- 2.4 Vergelijking van YOLO-Modellen voor Dakschade-Detectie . . . 9
  - 2.4.1 YOLO: Real-Time Geschikt voor Dakschade-Detectie . . . 9
  - 2.4.2 Evolutie van YOLO-Modellen (YOLOv4 – YOLOv9) . . . 9
- 2.5 Streaming van Dronebeelden . . . 11
  - 2.5.1 RTSP (Real-Time Streaming Protocol) . . . 11
  - 2.5.2 WebRTC . . . 12
  - 2.5.3 GStreamer . . . 12
  - 2.5.4 RTMP (Real-Time Messaging Protocol) . . . 12

3 Methodologie 13
- 3.1 Fase 1: Vooronderzoek en Requirements Analyse . . . 13
- 3.2 Fase 2: Dataset-onderzoek en Data Verzameling . . . 14
  - 3.2.1 Dataset-onderzoek . . . 14
  - 3.2.2 Eigen dataverzameling . . . 14
- 3.3 Fase 3: Model-selectie en Vergelijking . . . 15
- 3.4 Fase 4: Implementatie en Ontwikkeling . . . 15
- 3.5 Fase 5: Evaluatie en Analyse . . . 16

3.6 Fase 6: Validatie en Documentatie 16

4 Dataverzameling en Annotatie 18
4.1 Doelstelling van de dataverzameling 18
4.2 Opbouw van de dataset 18
4.3 Kenmerken van de dataset 19
4.4 Augmentatie via Roboflow 20
4.5 Samenvattend 21

5 Ontwikkeling en Training van Objectdetectiemodellen 22
5.1 Inleiding 22
5.2 Vergelijking van YOLO-modellen 22
5.2.1 YOLOv5s 22
5.2.2 YOLOv8s 23
5.2.3 Tussenconclusie 23
5.3 Explainable AI met YOLOv8_Explainer 24
5.3.1 Doel van Explainability 24
5.3.2 Over het YOLOv8_Explainer-pakket 24
5.3.3 Gekozen methode: EigenCAM 25
5.3.4 Toepassing binnen dit project 25
5.3.5 Referentie 26
5.4 Training van YOLOv5 op ongebalanceerde dataset 26
5.4.1 Trainingsverloop en verliescurves 26
5.4.2 Testresultaten op externe dataset 27
5.4.3 Heatmapanalyse 28
5.5 Training van YOLOv8 op ongebalanceerde dataset 29
5.5.1 Trainingscurves 30
5.5.2 Test op Externe Dataset 30
5.5.3 Heatmapanalyse 31
5.5.4 Reflectie op Prestaties 32
5.6 Balancing met YOLOWeightedDataset 33
5.6.1 Implementatie van YOLOWeightedDataset 34
5.7 Training van YOLOv8 met gebalanceerde dataset 36
5.7.1 Trainingsresultaten met SQRT-balancing 36
5.7.2 Resultaten op de externe testset 36
5.7.3 Verbeterde visuele aandacht volgens heatmaps 37
5.7.4 Conclusie SQRT-balancing 38
5.7.5 Visualisatie van blootstelling per klasse 39
5.8 Vergelijkende evaluatie op gevarieerde dataset 39
5.8.1 Prestatiemetrieken per modelversie 40
5.8.2 Analyse van confidence-patronen 40
5.8.3 Vergelijkende analyse en conclusie 41

6 Real-time Streaming en Detectieproces **43**
6.1 Doelstelling van het streaming- en detectieproces . . . 43
6.2 Waarom RTMP en hoe de lokale server werd opgezet . . . 43
6.3 Hoe het detectiescript werkt op een RTMP-stream . . . 44
6.4 Wat YOLO precies doet tijdens inference . . . 45
6.5 Patch-Based Inference als uitbreiding . . . 45
6.6 Aided Hyper Inference . . . 45
6.6.1 Technische implementatie . . . 46
6.6.2 Voordelen voor dakschade-detectie . . . 46
6.6.3 Implementatiemogelijkheden voor het dakschade-detectiesysteem 46
6.6.4 Praktische toepassing . . . 47
6.7 Conclusie . . . 47

7 Praktijktest en Evaluatie **48**
7.1 Doel van de praktijktest . . . 48
7.2 Testopstelling en aanpak . . . 48
7.3 Testresultaten en detecties . . . 49
7.3.1 Prestatie metingen . . . 49
7.4 Problemen en beperkingen . . . 52
7.4.1 Materiaalverschil als grote uitdaging . . . 52
7.4.2 Technische beperkingen . . . 52
7.4.3 Praktische problemen . . . 52
7.5 Analyse en vervolgstappen . . . 53
7.5.1 Positieve bevindingen . . . 53
7.5.2 Belangrijke inzichten . . . 53
7.5.3 Aanbevelingen voor vervolgonderzoek . . . 53
7.6 Conclusie . . . 53

8 Conclusie **54**
8.1 Beantwoording van de hoofdonderzoeksvraag . . . 54
8.2 Prestaties en bevindingen . . . 54
8.3 Technische doorbraak bewezen . . . 54
8.4 Relevantie en impact . . . 55

9 Verdere uitbreiding **56**
9.1 Datasetuitbreiding en diversificatie . . . 56
9.2 Patch-based inference implementatie . . . 56
9.3 Streaming en hardware optimalisatie . . . 57
9.4 Gebruikersinterface en workflow-integratie . . . 57
9.5 Commerciële haalbaarheid en implementatie . . . 57
9.6 Conclusie . . . 57

A Onderzoeksvoorstel **58**
A.1 Inleiding . . . 58
A.1.1 Doelgroep en Probleemstelling . . . 58
A.1.2 Centrale Onderzoeksvraag . . . 58
A.1.3 Deelvragen . . . 59
A.1.4 Keuze voor Implementatie-aanpak . . . 59
A.1.5 Onderzoeksdoelstelling . . . 60
A.2 Literatuurstudie . . . 60
A.2.1 Objectdetectie en Computer Vision . . . 60
A.2.2 Open-Source versus Commerciële Oplossingen . . . 60
A.2.3 Machine Learning en Schadedetectie . . . 61
A.2.4 Objectdetectie in Verwante Domeinen . . . 61
A.2.5 Toepasbaarheid voor Kleinschalige Bedrijven . . . 61
A.3 Methodologie . . . 61
A.4 Verwacht resultaat en conclusie . . . 63

Bibliografie **66**

Lijst van figuren

2.1 Vergelijking tussen R-CNN en YOLO detectiemethoden. 7
2.2 Illustratie van het YOLO detectiemodel. 8

4.1 Voorbeelden van verschillende schadetypen: (a) Chipped Shingle, (b) Hail Impact , (c) Algemene schade dakschade 18
4.2 Voorbeelden van daktypen in de dataset: bitumineuze dakshingles aangevuld met dakpannen 19
4.3 Klassenverdeling in de dataset voor augmentatie (totaal 2.766 beelden). De onbalans toont de dominantie van algemene 'damage' labels en 'Hail Impact', terwijl specifieke schadetypen zoals 'Big-Damage' ondervertegenwoordigd zijn. 20

5.1 YOLOv5 trainingscurves 26
5.2 YOLOv5 confidence vs. ground truth 27
5.3 YOLOv5 succesvolle detectie heatmap 28
5.4 YOLOv5 gemiste detectie heatmap 29
5.5 YOLOv8 trainingsgrafieken 30
5.6 YOLOv8 confidence vs. ground truth 31
5.7 YOLOv8 succesvolle detectie 32
5.8 YOLOv8 gemiste detectie 32
5.9 Vergelijking dataloader balancing 34
5.10 YOLOv8 SQRT-balanced training 36
5.11 Confusion matrix SQRT-balanced model 37
5.12 SQRT-balanced succesvolle detectie 37
5.13 SQRT-balanced verbeterde focus 38
5.14 Vergelijking klasblootstelling 39
5.15 YOLOv5 confidence op gevarieerde dataset 40
5.16 YOLOv8 ongebalanceerd confidence op gevarieerde dataset 40
5.17 YOLOv8 gebalanceerd confidence op gevarieerde dataset 41

6.1 MediaMTX server actief 44
6.2 DJI streaming instellingen 44
6.3 MediaMTX ontvangt drone stream 44

7.1 Dji mini 2 49

7.2 Live interface met real-time herkenningsresultaten tijdens de praktijk-test 50
7.3 dak variatie met confidence scores tussen 0.32-0.74 50
7.4 Degranulation detectie 1 met subtiele schadepatronen 51
7.5 Degranulation detectie 2 met subtiele schadepatronen 52

Lijst van tabellen

5.1 YOLOv5 prestatie-indicatoren 28
5.2 YOLOv8 prestatie-indicatoren 30
5.3 Vergelijkende prestaties verschillende modellen 40

7.1 RTMP Stream Latency Metingen 50
7.2 YOLO Inference Prestaties Vergelijking 51

Lijst van codefragmenten

5.1 YOLOv8_Explainer installatie 24
5.2 YOLOv8_Explainer heatmap generatie 25
5.3 YOLOv8 heatmap generatie code 31
5.4 YOLOWeightedDataset klasse definitie 34
5.5 YOLOWeightedDataset sampling mechanisme 35

1

Inleiding

In de moderne bouwindustrie is het inspecteren van daken een cruciale maar tegelijkertijd gevaarlijke en arbeidsintensieve taak. Traditionele dakinspecties vereisen dat inspecteurs fysiek op het dak klimmen, wat aanzienlijke veiligheidsrisico's met zich meebrengt; volgens het Center for Construction Research and Training (CPWR) zijn valpartijen vanaf daken verantwoordelijk voor meer dan één op drie (36,4%) dodelijke ongevallen in de bouwsector in 2020 (Brown e.a., 2021a). Daarnaast zijn conventionele inspecties tijdrovend, kostbaar en soms beperkt in nauwkeurigheid door menselijke factoren.
De opkomst van drones biedt een revolutionaire oplossing voor deze uitdagingen. Drones kunnen een volledig dak in een fractie van de tijd scannen die een manuele inspectie vergt, terwijl ze tegelijkertijd honderden hoogwaardige beelden of zelfs een volledig 3D-model genereren (Laminar, 2022). Het combineren van deze drone-technologie met geavanceerde objectdetectie-algoritmen, in het bijzonder de YOLO (You Only Look Once) familie, opent nieuwe mogelijkheden voor snelle, veilige en nauwkeurige dakinspecties.
Deze bachelorproef onderzoekt hoe een real-time, drone-gebaseerd systeem voor automatische dakschade-detectie kan worden ontwikkeld en geïmplementeerd, waarbij de focus ligt op het optimaliseren van de balans tussen nauwkeurigheid en verwerkingssnelheid op een draagbaar computersysteem.

1.1. Probleemstelling

Ondanks de voordelen van drone-gebaseerde inspecties, blijft het handmatig analyseren van de verzamelde beelden een tijdrovend proces dat specifieke expertise vereist. Dakexperts moeten nog steeds urenlang door honderden of duizenden beelden scrollen om schade te identificeren, wat de efficiëntiewinst van de drone-technologie beperkt. De huidige praktijk omvat meestal post-flight analyse, waar-

bij de beelden pas na de inspectievlucht worden beoordeeld, wat betekent dat aanvullende opnames van verdachte gebieden vaak een tweede vlucht vereisen.
De doelgroep voor dit onderzoek bestaat uit dakdekkers, bouwkundige inspecteurs, verzekeringsinspecteurs en dakonderhoudsbedrijven die drones inzetten voor inspecties. Deze professionals hebben behoefte aan een systeem dat niet alleen de veiligheid verhoogt en de inspectietijd verkort, maar ook de nauwkeurigheid en consistentie van schadedetectie verbetert.
Het kernprobleem dat dit onderzoek aanpakt is de integratie van real-time beeldverwerking en schadedetectie in het drone-inspectieproces, zodat verdachte gebieden onmiddellijk kunnen worden geïdentificeerd en nader onderzocht tijdens dezelfde vlucht, wat de efficiëntie en effectiviteit van drone-inspecties aanzienlijk zou verbeteren.

1.2. Onderzoeksvraag

De hoofdonderzoeksvraag van deze bachelorproef luidt:
Hoe kan een real-time dakschade-detectiesysteem worden ontwikkeld dat gebruikmaakt van drone-beelden en YOLO-objectdetectie op een draagbaar computersysteem met optimale balans tussen verwerkingssnelheid en detectienauwkeurigheid?
Deze hoofdvraag kan verder worden opgesplitst in de volgende deelvragen:

- Welke variant van het YOLO-algoritme (YOLOv4 tot YOLOv9) biedt de beste balans tussen verwerkingssnelheid en detectienauwkeurigheid voor dakschadedetectie op een draagbaar computersysteem (gaming laptop)?
- Welk streamingprotocol (RTSP, WebRTC, GStreamer) is het meest geschikt voor het verzenden van drone-beelden met minimale latentie naar het verwerkingssysteem?
- Welke preprocessing-technieken kunnen worden toegepast om de beeldkwaliteit te verbeteren en de detectienauwkeurigheid te verhogen zonder de real-time verwerkingscapaciteit aanzienlijk te beïnvloeden?
- Hoe kan het systeem worden geoptimaliseerd voor het detecteren van specifieke soorten dakschade (zoals gebroken dakpannen, lekkages, ontbrekende shingles) met minimale fout-positieven en fout-negatieven?

1.3. Onderzoeksdoelstelling

Het doel van dit onderzoek is het ontwikkelen en valideren van een prototype voor een real-time dakschade-detectiesysteem dat drone-beelden verwerkt met YOLO-objectdetectiealgoritmen op een draagbaar computersysteem. Het beoogde prototype moet aan de volgende criteria voldoen:

- Het systeem moet in staat zijn om video-streams van een drone in real-time te ontvangen en te verwerken met een maximale end-to-end latentie van 1 seconde.
- Het systeem moet ten minste drie verschillende soorten dakschade kunnen detecteren (bijv. gebroken dakpannen, ontbrekende elementen, scheuren) met een minimale nauwkeurigheid (mAP) van 75%.
- Het model moet kunnen draaien op een draagbare gaming laptop (bijv. ASUS ROG Zephyrus G15 met NVIDIA RTX 3070 GPU) met minimaal 15 FPS verwerkingssnelheid bij een invoerresolutie van 640x640 pixels.
- Het systeem moet een gebruiksvriendelijke interface bieden die real-time detectieresultaten visualiseert en de mogelijkheid biedt om verdachte gebieden te markeren voor nadere inspectie.

Het eindresultaat zal bestaan uit een werkend prototype, technische documentatie, gedetailleerde evaluatie van de prestaties (snelheid, nauwkeurigheid, betrouwbaarheid), en een vergelijkende analyse van de verschillende geteste YOLO-varianten en streamingprotocollen.

1.4. Opzet van deze bachelorproef

De rest van deze bachelorproef is als volgt opgebouwd:
In Hoofdstuk 2 wordt een overzicht gegeven van de stand van zaken binnen het onderzoeksdomein, op basis van een literatuurstudie. Hierin wordt dieper ingegaan op drone-technologie in de bouwsector, objectdetectie-algoritmen met nadruk op de YOLO-familie, en streamingprotocollen voor real-time beeldverwerking.
In Hoofdstuk 3 wordt de methodologie toegelicht en worden de gebruikte onderzoekstechnieken besproken om een antwoord te kunnen formuleren op de onderzoeksvragen. Dit omvat de ontwikkeling van een testopstelling, dataverzameling, modeltraining en evaluatiemethoden.
In Hoofdstuk 4 wordt de dataverzameling en annotatie beschreven, waarbij de opbouw van de dataset, de kenmerken ervan, en de toegepaste augmentatietechnieken via Roboflow worden toegelicht.
In Hoofdstuk 5 wordt de ontwikkeling en training van objectdetectiemodellen gedetailleerd beschreven, inclusief de vergelijking tussen YOLOv5 en YOLOv8, de implementatie van explainable AI technieken, en de toepassing van SQRT-balancing voor ongebalanceerde datasets.
In Hoofdstuk 6 wordt het real-time streaming en detectieproces uitgelegd, waarbij de RTMP-streamingopstelling, de werking van het detectiescript, en mogelijke uitbreidingen zoals patch-based inference worden besproken.

In Hoofdstuk 7 worden de praktijktest en evaluatie gepresenteerd, waarbij de prestaties van het complete systeem onder realistische omstandigheden worden getoetst en geanalyseerd.
In Hoofdstuk 8, tenslotte, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvragen. Daarbij wordt ook de relevantie en impact van het onderzoek besproken.
In Hoofdstuk 9 worden de mogelijkheden voor verdere uitbreiding geschetst, inclusief datasetuitbreiding, patch-based inference implementatie, en aanbevelingen voor commerciële haalbaarheid.

2

Stand van zaken

Dit hoofdstuk bevat een literatuurstudie over het gebruik van drones en kunstmatige intelligentie bij dakinspecties. Het bouwt verder op de inleiding en gaat dieper in op het centrale onderzoeksonderwerp. Het doel is om de lezer een duidelijk beeld te geven van de huidige stand van zaken in het veld. Ook wie nog niet vertrouwd is met het onderwerp, zou na dit hoofdstuk voldoende achtergrond moeten hebben om de rest van het werk vlot te kunnen volgen.

2.1. Drone-Technologie in de Bouwsector

Drones worden steeds vaker gebruikt in de bouw- en infrastructuursector, vooral voor inspectie en monitoring. Ze zijn inzetbaar voor uiteenlopende taken zoals het in kaart brengen van bouwplaatsen, volumemetingen en het inspecteren van moeilijk toegankelijke plekken zoals hoogspanningslijnen, bruggen en daken (Brown e.a., 2021a).

Bij schade-inspecties van gebouwen en daken bieden drones belangrijke voordelen ten opzichte van klassieke methodes. Een ervaren dronepiloot kan de buitenschil van een gebouw in een fractie van de tijd scannen die een manuele inspectie vergt (Engineering, 2019). In één vlucht kunnen honderden foto's of een volledig 3D-model van een dak worden vastgelegd, terwijl een manuele inspectie meer tijd vraagt en fysiek intensiever is (Laminar, 2022).

2.1.1. Voordelen van Drone-Inspecties

Drones verhogen de veiligheid doordat inspecteurs niet op hoogte moeten werken. Het risico op vallen of andere ongevallen wordt geëlimineerd, wat niet alleen levens beschermt maar ook de aansprakelijkheid en verzekeringskosten voor bedrijven vermindert (Engineering, 2019). Volgens een studie van het Center for Construction Research and Training (CPWR) zijn valpartijen vanaf daken verantwoordelijk voor

meer dan één op drie (36,4%) dodelijke ongevallen in de bouwsector in 2020 (Brown e.a., 2021a).
Daarnaast dringen drones de kosten terug: een snelle vlucht vervangt uren handmatig werk en dure voorzieningen zoals steigers of hoogwerkers (Engineering, 2019). Drones kunnen bovendien zeer gedetailleerde beelden vastleggen (vaak tot 4K-resolutie) en met thermische camera's temperatuurverschillen opsporen die kunnen duiden op lekkages of isolatieproblemen (Hezaveh e.a., 2022).
Dit leidt tot hogere datakwaliteit en nauwkeuriger schade-detectie dan het menselijk oog vaak kan leveren. Ook kunnen digitale drone-beelden eenvoudig gedeeld en hergebruikt worden, bijvoorbeeld in rapportages of om second opinions te vragen (Engineering, 2019). Verder zijn drones wendbaar en bereiken ze plekken die voor mensen moeilijk of gevaarlijk toegankelijk zijn zoals onder solar panels of langs steile dakranden (Ukhwah e.a., 2019).

2.1.2. Nadelen en Uitdagingen

Het gebruik van drones brengt ook enkele beperkingen met zich mee. Ten eerste vergt het bedienen van drones en het verwerken van de data de nodige expertise. Professioneel dronegebruik vereist een erkende vergunning, zoals een FAA-licentie, en registratie bij de luchtvaartautoriteiten (Team, 2024).
Daarnaast zijn drones weersafhankelijk: harde wind, regen of sneeuw kunnen een vlucht verhinderen of de beeldkwaliteit aantasten (Laminar, 2022). De vliegtijd van professionele drones is meestal beperkt tot 20 à 30 minuten, waardoor grote inspecties over meerdere vluchten gespreid moeten worden. Ook bestaat er kans op storingen of crashes van de drone, wat verlies van duur materiaal kan betekenen (Laminar, 2022).
Privacy en beveiligingszorgen spelen eveneens: drones verzamelen gedetailleerde beelden die vertrouwelijk moeten worden behandeld, en er is een (klein) risico op hacking of toegang tot de drone of data (Mekdad e.a., 2023). Ten slotte kan niet alle schade alleen visueel opgespoord wordenen zal fisieke interactie vereisen zoals schade onder zonnepanelen.
Toch worden drones steeds vaker gebruikt als inspectietool, en leveren ze meer return on investment op in bouwtoepassingen (Markets & Markets, 2022).

2.2. Objectdetectie en Computer Vision voor Schadeanalyse

Automatische dakschade-detectie steunt in de eerste plaats op objectdetectie in beelden. Objectdetectie-algoritmen analyseren afbeeldingen of videoframes en lokaliseren en classificeren specifieke objecten, zoals beschadigde dakpannen, ontbrekende shingles of scheuren (Girshick e.a., 2014). Dankzij deep learning herkennen moderne CNN's schadepatronen zoals hagelinslag of haarscheuren in beton,

ook wanneer die moeilijk zichtbaar zijn (Chen e.a., 2020).

2.2.1. Two-Stage vs One-Stage Detectors

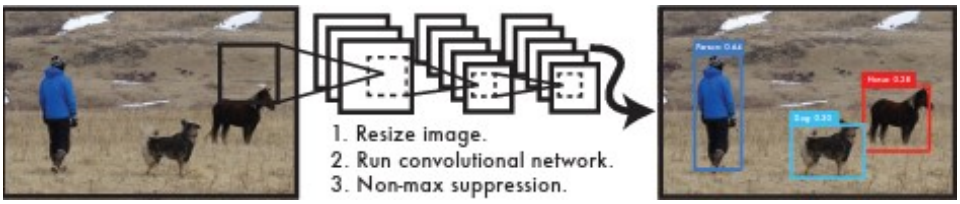
Objectdetectiemodellen worden vaak ingedeeld in two-stage en one-stage varianten:

- Two-stage modellen (zoals Faster R-CNN en Mask R-CNN) bieden hoge nauwkeurigheid, maar zijn relatief traag Girshick2014.
- One-stage modellen zoals YOLO verwerken beelden snel en zijn geschikt voor real-time toepassingen met drones (Redmon e.a., 2015).

De keuze voor het model hangt af van de context waarin het gebruikt wordt: real-time schade-analyse vereist een snelle one-stage detector zoals De YOLO modellen, erwijl bij een post flight analyse het gebruik van een nauwkeuriger two stage model zoals Mask R-CNN logischer zal zijn (Bochkovskiy e.a., 2020).

2.3. YOLO: Een Revolutionaire Aanpak in Objectdetectie

Het YOLO-algoritme (You Only Look Once) veranderde de aanpak van objectdetectie fundamenteel. Traditionele objectdetectiemethoden, zoals Region-Based Convolutional Neural Networks (R-CNNs), werken in twee stappen: eerst worden regio's in het beeld geselecteerd en vervolgens worden deze geclassificeerd (Girshick e.a., 2014). Dit proces verhoogt de nauwkeurigheid, maar introduceert ook vertragingsfactoren in de verwerking. Zoals geïllustreerd in Figuur 2.1 zien we dat YOLO objectdetectie fundamenteel anders benadert door het proces als één enkel regressieprobleem te beschouwen. Zoals Redmon et al. (2016) beschrijven in hun originele paper, herformuleert YOLO objectdetectie als een regressieprobleem in plaats van een classificatieprobleem. In plaats van classificatoren te hergebruiken voor detectie, gebruikt YOLO een enkel neuraal netwerk dat begrenzingsvakken en klasselabels direct voorspelt vanuit volledige beelden in één evaluatie (Redmon e.a., 2015).



Figuur 2.1: Vergelijking tussen R-CNN en YOLO detectiemethoden. (Bron: Redmon et al., 2016, Figuur 1)

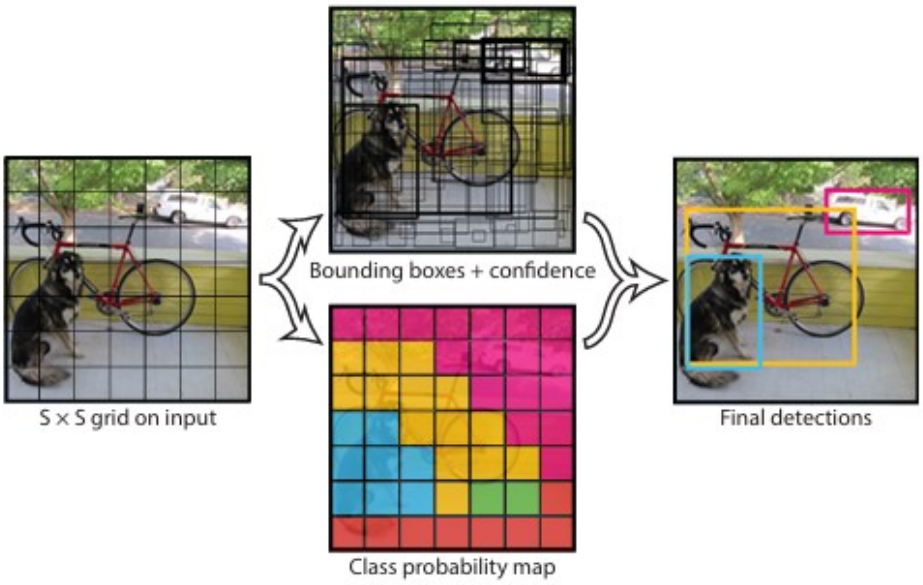
2.3.1. Werkingsprincipe van YOLO

Het werkingsprincipe van YOLO is gebaseerd op een rasterindeling en gelijktijdige voorspelling van objecten. Het model verwerkt het volledige beeld in één door-

gang en voorspelt daarbij zowel de locaties als de klassen van objecten. Het proces verloopt als volgt:

- Het invoerbeeld wordt opgedeeld in een raster van S × S cellen.
- Elke cel is verantwoordelijk voor het detecteren van een object als het middelpunt van dat object binnen die cel valt.
- Per cel voorspelt het model B begrenzingsvakken (bounding boxes) en een betrouwbaarheidsscore voor elk vak.
- Deze score weerspiegelt zowel de waarschijnlijkheid dat er een object aanwezig is, als de nauwkeurigheid van de voorspelde bounding box.
- Elke bounding box bestaat uit vijf waarden: x, y, w, h en betrouwbaarheid. De coördinaten x en y geven de positie van het vakcentrum aan ten opzichte van de grenzen van de cel.
- Elke cel voorspelt ook C klassewaarschijnlijkheden van het type $Pr(\text{klasse}_i \mid \text{object})$, met één waarde per objectklasse.

In het originele YOLO-model resulteert dit in een uitvoertensor van $S \times S \times (B \times 5 + C)$. Voor de PASCAL VOC dataset gebruikte men $S = 7$, $B = 2$ en $C = 20$, wat leidde tot een tensor van $7 \times 7 \times 30$. (Redmon e.a., 2015).



Figuur 2.2: Illustratie van het YOLO detectiemodel en rasterindeling. Het beeld wordt opgedeeld in een rooster en per cel worden begrenzingsvakken en klassewaarschijnlijkheden voorspeld (Redmon et al., 2016, Figuur 2).

De oorspronkelijke YOLO-architectuur bestond uit 24 convolutielagen gevolgd door 2 volledig verbonden lagen. De convolutielagen zijn verantwoordelijk voor het extraheren van visuele kenmerken uit het beeld. De volledig verbonden lagen vertalen deze kenmerken vervolgens naar concrete voorspellingen van bounding boxes en klassen.

2.4. Vergelijking van YOLO-Modellen voor Dakschade-Detectie

Gezien de behoefte aan real-time objectdetectie in drone-toepassingen, is de YOLO-familie (You Only Look Once) bijzonder relevant. YOLO-modellen behoren tot de snelste detectoren en hebben door de jaren heen aanzienlijke verbeteringen doorgemaakt in nauwkeurigheid en efficiëntie. Deze sectie vergelijkt de evolutie van YOLO van versie 4 tot en met de meest recente ontwikkelingen (YOLOv9), met specifieke aandacht voor hun geschiktheid voor dakschade-detectie op een draagbaar systeem zoals een gaming-laptop, bijvoorbeeld de ASUS ROG Zephyrus G15.

2.4.1. YOLO: Real-Time Geschikt voor Dakschade-Detectie

YOLO verwerkt het volledige beeld in één doorgang en voorspelt rechtstreeks de locaties en klassen van objecten. Dit maakt het aanzienlijk sneller dan oudere detectiemethoden, zoals Faster R-CNN, die meerdere stappen vereisen om objecten correct te identificeren (Redmon e.a., 2016). YOLOv1 behaalde in 2016 al 45 FPS op een Nvidia Titan X-GPU, al was de nauwkeurigheid lager dan bij latere versies (Redmon e.a., 2015). YOLOv4 verhoogt de nauwkeurigheid sterk zonder veel in te boeten op snelheid (Bochkovskiy e.a., 2020). De modellen zijn beschikbaar in verschillende groottes, van compacte versies zoals YOLO-Tiny of YOLO-Nano, die geschikt zijn voor embedded systemen, tot grotere architecturen die vergelijkbare prestaties leveren als complexere modellen zoals Faster R-CNN en Mask R-CNN (Viso.ai, 2024). De combinatie van hoge verwerkingssnelheid en eenvoudige integratie maakt YOLO bijzonder geschikt voor real-time drone-inspecties waarbij schade meteen kan worden gesignaleerd, zodat inspecteurs tijdig kunnen bijsturen of aanvullende beelden kunnen opnemen.

2.4.2. Evolutie van YOLO-Modellen (YOLOv4 – YOLOv9)

YOLOv4 (2020)

YOLOv4, ontwikkeld door Bochkovskiy et al. (2020), introduceerde optimalisaties zoals CSPDarknet53, Spatial Pyramid Pooling (SPP), PANet, Mosaic data augmentation en CIoU-loss. Op de COCO test-dev dataset behaalde het model een mAP van 43,5% bij 62 FPS op een Tesla V100 met 608×608 input (Bochkovskiy e.a., 2020). Andere varianten, zoals yolov4-p5 en yolov4-csp-x-swish, behaalden volgens Iriemon (2020) een hogere mAP tot 54%, maar met een lagere snelheid van 43 tot 50 FPS, afhankelijk van de gebruikte instellingen. (Iriemon, 2020).

YOLOv5 (2020/21)

YOLOv5, uitgebracht door Ultralytics, verscheen kort na YOLOv4. Het model werd niet in een academische paper gedocumenteerd, maar bracht significante voordelen met zich mee. Het was geschreven in PyTorch, bood verschillende modelgroottes (s, m, l, x) en verbeterde de efficiëntie van inferentieprocessen. Volgens benchmarks van Roboflow haalde de kleinste variant (YOLOv5s) een inferentietijd van slechts ~0,007 seconden per beeld (140 FPS) op high-end GPU's (Nelson & Solawetz, 2020). In tests met batch-inferentie scoorde YOLOv5 ook beter dan YOLOv4, hoewel het verschil kleiner werd bij batch = 1, waar YOLOv4 ~30 FPS haalde en YOLOv5 rond de 10 FPS op CPU bleef hangen (Nelson & Solawetz, 2020). Bij inference in batchvorm presteerde YOLOv5 sneller dan YOLOv4, al lag het verschil kleiner bij batch = 1. Op CPU haalde YOLOv5 toen ongeveer 10 FPS tegenover 30 FPS voor YOLOv4. Over het geheel genomen bood YOLOv5 een betere efficiëntie, met een vergelijkbare of iets hogere mAP. In een vergelijkende studie haalde YOLOv5-l een nauwkeurigheid van 85,65% en een mAP van 78,9%, iets lager dan de topwaarden van YOLOv7 (Singh e.a., 2024).

YOLOv6 (2022)

YOLOv6 werd in 2022 uitgebracht door Meituan als een industriële variant, gericht op edge computing. Het model introduceerde een nieuwe backbone (EfficientRep), een decoupled head en technieken uit PP-YOLOE, zoals SIoU en Task Alignment Learning (LearnWithARobot, 2024). Volgens benchmarks is YOLOv6 ongeveer 15% trager dan YOLOv7, en behaalt het model ook een lagere gemiddelde precisie. In tests haalde YOLOv6-l een mAP van 72,32%, vergeleken met 78,9% voor YOLOv5-l en 79,0% voor YOLOv7 (Singh e.a., 2024). Ondanks deze verschillen blijft YOLOv6 relevant in contexten waar optimalisaties zoals TensorRT-integratie van Meituan een vereiste zijn.

YOLOv7 (2022)

YOLOv7, uitgebracht door Wang et al. (2022), betekende een duidelijke vooruitgang in zowel snelheid als nauwkeurigheid binnen de YOLO-reeks. Het model introduceerde de Extended ELAN-architectuur, verbeterde trainingstechnieken en meerdere schaalvarianten (YOLOv7-Tiny tot YOLOv7-E6E). Op de COCO test-dev dataset behaalde het een gemiddelde precisie (AP) van 56,8% bij een snelheid van 84 FPS op een Tesla V100 met 640×640 input (Wang e.a., 2023). Dit was een aanzienlijke verbetering ten opzichte van YOLOv5 en YOLOv4. In onafhankelijke tests werd vastgesteld dat YOLOv7 ongeveer 20% sneller was dan YOLOv5 bij hogere nauwkeurigheid (Viso.ai, 2024). Door deze combinatie van nauwkeurigheid en snelheid is YOLOv7 geschikt voor real-time toepassingen zoals drone-inspecties, zeker wanneer het op krachtige GPU-laptops draait.

YOLOv8 (2023)

YOLOv8 werd in januari 2023 uitgebracht door Ultralytics als een geoptimaliseerde opvolger van YOLOv5. Het model ondersteunt detectie, segmentatie en classificatie in één framework (Wang e.a., 2023). Volgens benchmarks van Stereolabs presteert YOLOv8 beter dan zowel YOLOv5 als YOLOv7 bij gelijke modelgroottes: bijvoorbeeld haalt YOLOv8m een mAP van 50,2% tegenover 45,4% voor YOLOv5m, en overtreft YOLOv8x ook YOLOv7x (53,9% vs 52,9%) (Stereolabs, 2023). YOLOv8-Nano overtrof YOLOv5n in nauwkeurigheid terwijl het een vergelijkbare snelheid behield (Stereolabs, 2023). In de praktijk draaien de kleinere versies (YOLOv8-s of -m) vlot real-time tussen 30 en 60 FPS, afhankelijk van de gebruikte GPU.

YOLOv9 en verder (2024)

YOLOv9 werd begin 2024 gelanceerd en bouwt voort op eerdere versies met nieuwe technieken zoals Programmable Gradient Information (PGI), Generalised Efficient Layer Aggregation (GELAN), en optimalisaties in het gebruik van anchorboxen (OpenSistemas, 2024). Deze technieken verkleinen het risico op informatieverlies en maken het model geschikt voor zowel lichte als zware toepassingen. Volgens benchmarks op de MS COCO dataset biedt YOLOv9 een 49% reductie in parameters en 43% minder rekentijd, met een stijging van 0,6% in nauwkeurigheid ten opzichte van YOLOv8 (Yaseen, 2024). Ondanks dat sommige bronnen zoals OpenSistemas het model eerder beschrijven als een “revolutionaire doorbraak”yolov9 paper, is het belangrijk te noteren dat peer-reviewed validatie pas sinds eind 2024 beschikbaar is.

2.5. Streaming van Dronebeelden

Voor real-time verwerking is het essentieel dat de videostream van de drone met zo weinig mogelijk vertraging bij de verwerkende laptop terechtkomt. De meeste moderne drones, zoals die van DJI, maken gebruik van interne transmissiesystemen die de beelden verzenden naar een controller of smartphone, met vertragingen tussen 200 en 500 milliseconden (Markets & Markets, 2022). Om de videofeed beschikbaar te maken voor een custom detectiesysteem op een laptop, bestaan er verschillende gangbare methoden.

2.5.1. RTSP (Real-Time Streaming Protocol)

RTSP is een protocol dat veel wordt gebruikt voor IP-camera’s en videostreaming binnen lokale netwerken. Het werkt samen met RTP (Real-Time Transport Protocol) om videodata frame per frame te verzenden. RTSP is relatief eenvoudig op te zetten en kan, met de juiste configuratie, vertragingen onder de 500 milliseconden bereiken (Team, 2024). Een drone met een camera die RTSP-streaming ondersteunt (of een aan boord geplaatste computer die de camerafeed als RTSP-server uitstuurt) kan zo live video beschikbaar stellen aan de laptop. De laptop hoeft dan

enkel een RTSP client (bv. VLC of OpenCV) te hebben om frame voor frame de stream binnen te halen voor analyse (Robotics, 2021).

2.5.2. WebRTC

WebRTC is ontworpen voor peer-to-peer communicatie met bijzonder lage vertraging. In geschikte netwerkomstandigheden zijn vertragingen onder de 500 milliseconden mogelijk (Team, 2024). WebRTC bevat standaardfuncties zoals foutcorrectie, encryptie en dynamische aanpassing van bitrate wat nuttig is bij wisselende netwerkomstandigheden zoals bij drones. In sommige toepassingen fungeert een drone of on-board computer als WebRTC-peer, waarbij de laptop als tegenpartij direct een versleutelde stream ontvangt (Team, 2024). Het opzetten van een WebRTC-verbinding vereist componenten zoals een signaling server, NAT traversal (via STUN of TURN), en een media-encoder.

2.5.3. GStreamer

GStreamer is een veelzijdig multimedia-framework waarmee men op laag niveau streaming pipelines kan bouwen. Het wordt vaak gebruikt voor dronevideo vanwege de mogelijkheid om zelf encoders, transportprotocollen en buffers te configureren voor minimale vertraging (Robotics, 2021). Met GStreamer kan men bijvoorbeeld een pipeline opzetten die de camerastreaming van de drone pakt, waar een tegenhanger-pipeline de stream decodeert. Juist door deze directe aanpak is GStreamer in staat zeer lage latency te behalen. In toepassingen waar latency cruciaal is, kan GStreamer geconfigureerd worden om videobeelden vrijwel onmiddellijk door te sturen, met minimale buffering. In combinatie met aangepaste netwerkinstellingen zijn vertragingen van onder 100 milliseconden technisch haalbaar, al zijn deze sterk afhankelijk van hardware en configuratie. (Stereolabs, 2023).

2.5.4. RTMP (Real-Time Messaging Protocol)

RTMP werd oorspronkelijk ontwikkeld door Adobe voor het verzenden van audio, video en data tussen Flash-gebaseerde toepassingen. Vandaag wordt het protocol nog steeds gebruikt als open specificatie, met ondersteuning voor verschillende mediatypes zoals AMF, FLV en F4V, verzonden via het betrouwbare TCP-protocolaloman2015. In een studie van Aloman et al. (2015) werd RTMP getest in een mobiele Wi-Fi-omgeving, waarbij een smartphone via WowzaGoCoder live video streamde naar een Wowza-server. De onderzoekers simuleerden packet loss en vertraging bij videobitrates van 280 Kbps en 700 Kbpsaloman2015. Uit deze tests bleek dat RTMP in meerdere scenario's een hogere gebruikskwaliteit bood dan RTSP, gemeten via de Mean Opinion Score (MOS). In sommige gevallen presteerde het zelfs beter dan MPEG-DASH. De resultaten suggereren dat RTMP door het gebruik van TCP minder gevoelig is voor pakketverlies en daardoor stabielere prestaties biedt onder moeilijke netwerkcondities (Aloman e.a., 2015).

3

Methodologie

Dit hoofdstuk beschrijft de methodologische aanpak die is gehanteerd voor het ontwikkelen en evalueren van het real-time dakschade-detectiesysteem met drone-beelden. Het onderzoek is opgedeeld in vijf opeenvolgende fasen, elk met specifieke doelstellingen en deliverables. Deze fasering waarborgt een gestructureerde en systematische benadering van het onderzoeksprobleem.

3.1. Fase 1: Vooronderzoek en Requirements Analyse

Deze initiële fase, met een duur van 3 weken, was gericht op het verkrijgen van een diepgaand inzicht in de technische vereisten en mogelijkheden voor real-time dakschade-detectie.

Doelstelling: Vaststellen van technische haalbaarheid en selecteren van geschikte technologieën voor real-time implementatie.

Uitgevoerde activiteiten:

- Uitvoeren van een diepgaande literatuurstudie naar real-time objectdetectie-modellen, met specifieke focus op de YOLO-familie (v4 tot v9)
- Inventarisatie van beschikbare open-source tools en libraries geschikt voor real-time verwerking
- Opstellen van concrete evaluatiecriteria met focus op verwerkingssnelheid en hardware-eisen
- Analyse van de drone-hardware capaciteiten en beperkingen
- Onderzoek naar verschillende streaming-mogelijkheden tussen drone en computer (RTSP, WebRTC, GStreamer, RTMP)

Onderzoeksmethoden: Systematische literatuurreview, technische benchmarking van beschikbare tools, requirements engineering.

Deliverable: Gedetailleerd technisch rapport met evaluatiecriteria, geselecteerde tools en het aanbevolen detectiemodel voor real-time implementatie.

3.2. Fase 2: Dataset-onderzoek en Data Verzameling

Deze fase, met een duur van 4 weken, was gericht op het identificeren van beschikbare datasets en het opbouwen van een representatieve dataset voor dakschade-detectie.
Doelstelling: Verkrijgen van voldoende trainingsdata door onderzoek naar bestaande datasets en aanvullende dataverzameling waar nodig.
Uitgevoerde activiteiten:

3.2.1. Dataset-onderzoek

- Systematisch onderzoek naar open source datasets voor dakschade-detectie
- Evaluatie van bestaande datasets zoals:
  - Rooftop Solar Panel Detection datasets
  - Building damage assessment datasets van natuurrampen
  - Algemene aerial imagery datasets (xView, DOTA)
  - Roboflow datasets
- Analyse van dataset-kwaliteit, annotatiestypes en geschiktheid voor real-time detectie
- Vergelijking van beschikbare dakschadetypen in bestaande datasets

3.2.2. Eigen dataverzameling

- Verzamelen van drone-testbeelden onder verschillende omstandigheden (licht-omstandigheden, weersomstandigheden, daktypen)
- Ontwikkelen van een systematische aanpak voor real-time video-analyse
- Voorbereiden van trainingsdata voor het optimaliseren van real-time detectie, inclusief annotatie van dakschade
- Implementatie van data augmentatie technieken voor dataset-uitbreiding

Motivatie voor Roboflow: Uit het onderzoek bleek dat bestaande open source datasets ofwel te algemeen waren (geen specifieke focus op dakschade) met Roboflow als uitzondering. Er werd gebruik gemaakt van 2 merged Roboflow datasets aangevuld met eigen data.
Onderzoeksmethoden: Comparative dataset analysis, systematische dataverzameling, annotation guidelines development.
Deliverable: Dataset met minimaal 2000 gelabelde voorbeelden van dakschade en testscenario's voor real-time detectie, inclusief motivatie voor dataset-keuzes.

3.3. Fase 3: Model-selectie en Vergelijking

Deze fase, met een duur van 2 weken, was specifiek gericht op het vergelijken van verschillende YOLO-varianten voor real-time detectie.
Doelstelling: Bepalen van de optimale YOLO-variant voor real-time dakschade-detectie op basis van empirische vergelijking.
Uitgevoerde activiteiten:

- Implementatie van YOLOv5 en YOLOv8 voor vergelijkende analyse
- Training van beide modellen op identieke datasets
- Kwantitatieve vergelijking van precisie, recall en inference-snelheid
- Analyse van model-complexiteit en hardware-vereisten
- Implementatie van SQRT-balancing voor ongebalanceerde datasets
- Evaluatie van verschillende confidence thresholds

Onderzoeksmethoden: Controlled experimentation, statistical comparison, ablation studies.
Deliverable: Vergelijkend rapport met onderbouwde model-selectie en geoptimaliseerde hyperparameters.

3.4. Fase 4: Implementatie en Ontwikkeling

Deze kernfase van het onderzoek, met een duur van 5 weken, omvatte het implementeren van de real-time detectie-pipeline.
Doelstelling: Ontwikkelen van een werkend real-time detectiesysteem met het geselecteerde model.
Uitgevoerde activiteiten:

- Ontwikkeling van een Python-gebaseerde applicatie voor real-time videoverwerking
- Implementatie van het geselecteerde model met optimalisaties voor snelle verwerking
- Opzetten van een real-time evaluatieframework
- Opzetten van een video-streaming pipeline
- Integratie van alle componenten tot een coherent systeem

Gebruikte tools:

- Python 3.x als primaire programmeertaal
- OpenCV voor video-capture en beeldverwerking

- PyTorch voor model implementatie
- MediaMTX voor RTMP streaming
- Git voor versiebeheer

Onderzoeksmethoden: Agile development and iterative prototyping.
Deliverable: Werkende real-time applicatie met geoptimaliseerd detectiemodel.

3.5. Fase 5: Evaluatie en Analyse

Deze fase, met een duur van 3 weken, was gericht op het systematisch testen en evalueren van het ontwikkelde systeem.
Doelstelling: Kwantitatieve en kwalitatieve evaluatie van systeemprestaties onder verschillende omstandigheden.
Uitgevoerde activiteiten:

- Kwantitatieve analyse van real-time detectie-accuraatheid (precision, recall, mAP)
- Meting van latentie en framerate van de streaming pipeline
- Analyse van CPU/GPU-gebruik op de verwerkende computer
- Praktijktests onder verschillende omstandigheden (verschillende daktypen, weersomstandigheden)
- Vergelijking van prestaties tussen controlled environment en real-world testing

Onderzoeksmethoden: Performance benchmarking, statistical analysis, field testing.
Deliverable: Uitgebreid evaluatierapport met real-time performantie metrics en analyse van limitaties.

3.6. Fase 6: Validatie en Documentatie

Deze afsluitende fase, met een duur van 3 weken, was gericht op het valideren van de resultaten en het documenteren van het onderzoek.
Doelstelling: Valideren van resultaten en opstellen van complete documentatie voor reproduceerbaarheid.
Uitgevoerde activiteiten:

- Validatie van real-time prestaties
- Cross-validation van detectieresultaten
- Opstellen van technische documentatie

- Schrijven van implementatierichtlijnen
- Analyse van commerciële haalbaarheid en vervolgstappen

Onderzoeksmethoden: Result validation, technical documentation, feasibility analysis.
Deliverable: Complete documentatie inclusief installatiehandleiding en demonstratie van real-time detectie.

4

Dataverzameling en Annotatie

4.1. Doelstelling van de dataverzameling

Het doel van de dataverzameling binnen dit project is het samenstellen van een dataset die voldoende representatief is om een betrouwbaar deep learning-model te trainen voor dakschade-detectie met behulp van dronebeelden. De dataset moet het model in staat stellen om enerzijds schade te herkennen, en anderzijds de aard van die schade te specificeren via onderverdelingen zoals Chipped Shingle of Hail Impact (zie Figuur 4.1). Aangezien de toepassing bedoeld is voor real-time ondersteuning tijdens dakinspecties, moet de dataset ook robuust zijn tegen variaties in belichting, camerahoek en resolutie.



Figuur 4.1: Voorbeelden van verschillende schadetypen: (a) Chipped Shingle, (b) Hail Impact , (c) Algemene schade dakschade

4.2. Opbouw van de dataset

Voor dit project werd gebruik gemaakt van meerdere bronnen om tot een diverse en bruikbare dataset te komen:

Openbare datasets via Roboflow

Twee specifieke publieke datasets (damage-assessment en roof-inspection) die gericht zijn op dakschadeherkenning werden gedownload via het platform Roboflow. Deze datasets werden gekozen omdat zij de meeste variatie in schadesoorten bevatten vergeleken met andere beschikbare datasets. Hoewel er nog gespecialiseerde datasets bestaan die later kunnen gebruikt worden voor uitbreiding en balancering, bieden deze twee datasets de beste basis voor het huidige onderzoek. Roboflow bleek in dit proces een erg waardevol hulpmiddel: het platform biedt niet alleen een gebruiksvriendelijke interface voor labeling, maar ook automatische preprocessing en augmentatiemogelijkheden.

Aanvullende data via internet

Aanvullend beeldmateriaal werd verzameld via verschillende online bronnen, waaronder professionele dakwerk websites en Google Images. Deze beelden zorgen ervoor dat het model blootgesteld wordt aan een bredere variatie aan dakschades en omstandigheden, wat de realistische toepasbaarheid ten goede komt.



Figuur 4.2: Voorbeelden van daktypen in de dataset: bitumineuze dakshingles aangevuld met dakpannen

Het onderzoek richt zich specifiek op bitumineuze dakshingles (zie Figuur 4.2), een veelvoorkomende dakbedekking die gevoelig is voor verschillende vormen van weersinvloeden en slijtage.

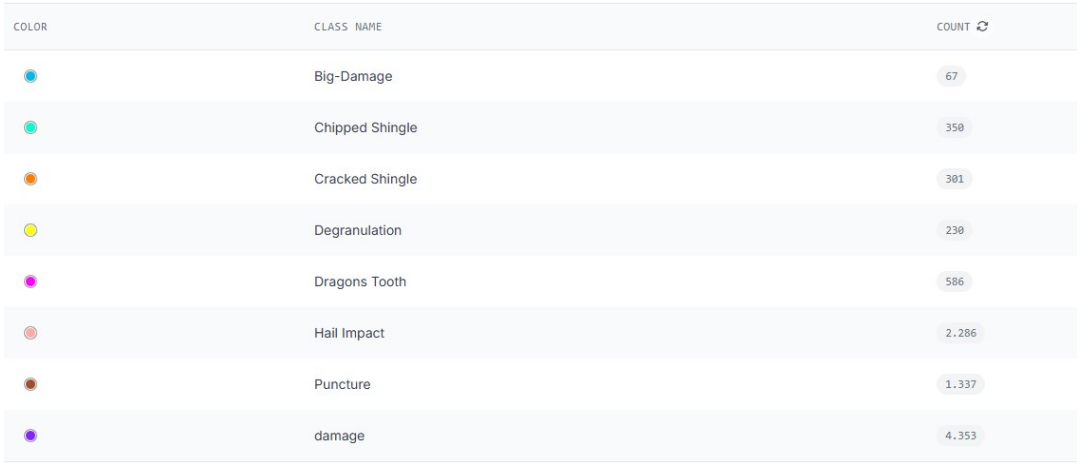
Manuele en AI-ondersteunde labeling

Alle beelden werden handmatig gelabeld via de Roboflow labeling-tool. Bij twijfel werd een algemene label *damage* toegekend. Enkel in gevallen waar de schade duidelijk te typeren was, werd een specifieke subklasse gebruikt zoals *Cracked Shingle* of *Puncture*. Hierdoor bevat de dataset relatief veel algemene schadelabels en minder specifieke labels, wat bewust gekozen is om eerst de detectie goed aan te leren en daarna als proof-of-concept subtypes te introduceren.

4.3. Kenmerken van de dataset

Ongebalanceerde verdeling

Door de labelingstrategie (eerst zeker zijn van schade, dan pas een subklasse kiezen), is de dataset ongebalanceerd. Het merendeel van de annotaties valt onder de brede klasse *damage* (4.353 instanties), gevolgd door *Hail Impact* (2.286) en andere specifieke schadetypen in kleinere aantallen (zie Figuur 4.3). Dit maakt de dataset geschikt voor schadeherkenning, maar beperkt de prestaties bij het onderscheiden van schadevormen. Toch biedt dit een goede basis voor verdere verfijning.



COLOR	CLASS NAME	COUNT
	Big-Damage	67
	Chipped Shingle	350
	Cracked Shingle	301
	Degranulation	230
	Dragons Tooth	586
	Hail Impact	2.286
	Puncture	1.337
	damage	4.353

Figuur 4.3: Klassenverdeling in de dataset voor augmentatie (totaal 2.766 beelden). De onbalans toont de dominantie van algemene 'damage' labels en 'Hail Impact', terwijl specifieke schadetypen zoals 'Big-Damage' ondervertegenwoordigd zijn.

Beelden zonder schade toegevoegd

Om het model te leren dat sommige daken geen schade vertonen, zijn ook beelden van onbeschadigde daken toegevoegd. Deze zijn bewust zonder annotatie gelaten, zodat het model leert dat niet elk dakdeel schade bevat.

4.4. Augmentatie via Roboflow

Om de dataset te versterken en het model te wapenen tegen overfitting, werd uitgebreid gebruik gemaakt van data-augmentatie. Roboflow biedt hiervoor een geïntegreerde toolset. De volgende augmentaties werden toegepast:

- Flip (horizontaal): beelden worden gespiegeld, zodat het model leert dat schade links of rechts kan voorkomen.
- Rotation (-20° tot +20°): kleine rotaties simuleren wisselende camerahoeken bij dronevluchten.
- Grayscale (15%): conversie naar zwart-wit op een deel van de dataset om robuustheid op te bouwen bij slechte kleurinformatie.

- Brightness (+/-15%): om te trainen op wisselende lichtomstandigheden (zon, bewolkt, schaduw).
- Noise (tot 0.1%): voegt visuele ruis toe om de invloed van compressie of bewegingsonscherpte te simuleren.

Door deze augmentatie-instellingen werd de dataset vergroot van 2.766 naar 7.198 beelden, verdeeld in training, validatie en testsets. Dit zorgde ervoor dat het model verschillende versies van dezelfde schade zag, met licht gewijzigde kenmerken.

4.5. Samenvattend

De uiteindelijke dataset is samengesteld uit publieke datasets (”damage-assessment- en ”roof-inspection”) en aanvullende internetbronnen, bevat zowel schade als niet-schadebeelden van bitumineuze dakshingles, en is handmatig geannoteerd via Roboflow. Door de bewuste keuze om bij twijfel enkel *damage* te labelen, fungeert de dataset als een solide basis voor detectie, met subtypes als een proof-of-concept. Roboflow speelde een centrale rol in het efficiënt samenstellen, labelen en augmenteren van de dataset van 2.766 naar 7.198 beelden.

5

Ontwikkeling en Training van Objectdetectiemodellen

5.1. Inleiding

Dit hoofdstuk behandelt de ontwikkeling en evaluatie van YOLO-detectiemodellen voor dakschade-herkenning. Na de dataverzameling en annotatie uit hoofdstuk 4 wordt hier gefocust op de praktische implementatie en training van de modellen. Het hoofdstuk heeft drie doelstellingen:

1. Vergelijken van YOLOv5 en YOLOv8 op prestaties en verwerkingssnelheid
2. Analyseren van modelgedrag via explainable AI technieken
3. Optimaliseren van training door databalancing voor ondervertegenwoordigde klassen

De resultaten vormen de basis voor de keuze van het definitieve model voor het real-time detectiesysteem.

5.2. Vergelijking van YOLO-modellen

In het kader van dit onderzoek werd gekozen om twee versies van het YOLO-framework met elkaar te vergelijken: YOLOv5s en YOLOv8s. Beide modellen zijn ontworpen voor snelle objectdetectie en worden veel gebruikt in toepassingen waar real-time analyse vereist is, zoals bij drone-inspecties.

5.2.1. YOLOv5s

YOLOv5s is een lichtgewicht model dat sinds 2020 breed wordt ingezet door de computer vision-community. Het is gebaseerd op PyTorch, en valt op door zijn

snelle inference-tijd en lage rekeneisen. De 's' in YOLOv5s staat voor small, wat betekent dat dit model bedoeld is voor toepassingen met beperkte rekenkracht of waar snelheid cruciaal is.
Tijdens de training op de dakschade-dataset (50 epochs) toonde YOLOv5s een relatief stabiel leerproces, maar met beperkte prestaties bij het herkennen van zeldzame schadeklassen. Vooral bij ongebalanceerde trainingdata had YOLOv5s de neiging om dominante klassen zoals 'damage' en 'Hail Impact' te bevoordelen.
Het model bevatte 157 lagen en ongeveer 7 miljoen parameters. Op een mid-range GPU draaide het vlot en bereikte een inference-tijd van minder dan 20 ms per afbeelding. De totale mAP-score na training bleef echter beperkt tot ongeveer 0.32.

5.2.2. YOLOv8s

YOLOv8 werd ontwikkeld door Ultralytics als een moderne versie van het YOLO-framework. Het model maakt gebruik van een vernieuwde backbone en moderne trainingsconcepten zoals DFL-loss (Distribution Focal Loss). Het biedt ook native ondersteuning voor export naar diverse deploymentformats, wat het ideaal maakt voor integratie in verschillende omgevingen.
Bij een vergelijkbare training van 50 epochs op dezelfde dataset haalde YOLOv8 een betere mAP van ongeveer 0.34 en hogere recall-waarden. De loss-curves daalden sneller en waren stabieler, wat wijst op een efficiënter leerproces.
Qua complexiteit telt YOLOv8s ongeveer 11 miljoen parameters en 72 lagen. Ondanks de toegenomen grootte verwerkte het model beelden sneller dan verwacht. Op een Tesla T4 GPU werd een inference-pipeline gemeten van:

- Preprocessing: 0.6 ms
- Inference: 7.0 ms
- Postprocessing: 6.3 ms

Dit komt overeen met een theoretische verwerkingssnelheid van meer dan 140 FPS, waarmee YOLOv8s niet alleen accurater, maar ook performanter bleek voor real-time dakinspectietoepassingen.

5.2.3. Tussenconclusie

De vergelijking tussen YOLOv5s en YOLOv8 toont aan dat de nieuwere YOLOv8-architectuur betere prestaties levert op zowel detectienauwkeurigheid als inferencesnelheid. Bovendien is YOLOv8 beter afgestemd op moderne workflows. Op basis hiervan werd besloten om YOLOv8 als hoofdmodel te behouden voor verdere ontwikkeling.

5.3. Explainable AI met YOLOv8_Explainer

Kwantitatieve prestatie-indicatoren zoals precision, recall en mAP zijn waardevol voor het beoordelen van modeleffectiviteit, maar ze verklaren niet waarom een model bepaalde beslissingen neemt. Voor dakschade-inspectie, waar AI-modellen financiële of structurele risico's beoordelen, is deze transparantie belangrijk. Dit onderzoek gebruikt daarom explainable AI (XAI) technieken, specifiek het open-source pakket YOLOv8_Explainer.

5.3.1. Doel van Explainability

XAI-technieken werden ingezet voor verschillende doeleinden:

- Diagnostisch gebruik: nagaan of het model focust op de juiste schadezones
- Begrijpbaarheid: foutdetecties verklaren (bijvoorbeeld waarom het model schade detecteert op een voet)
- Vertrouwen: eindgebruikers zoals dakwerkers inzicht geven in classificatiebeslissingen
- Debugging: bias of artefacten in trainingsdata identificeren

5.3.2. Over het YOLOv8_Explainer-pakket

De tool YOLOv8_Explainer is een open-source bibliotheek beschikbaar op GitHub (Spritan, 2024) die gebruik maakt van geavanceerde CAM-methodes (Class Activation Maps) om visuele verklaringen te genereren voor YOLOv8-resultaten. De tool is eenvoudig te integreren via:

```
pip install YOLOv8-Explainer
```

Codefragment 5.1: Installatie van YOLOv8_Explainer pakket

Eenmaal geïnstalleerd kunnen er heatmaps gegenereerd worden die aanduiden waar in een afbeelding het model activatie toont bij detectie:

```
from YOLOv8_Explainer import yolov8_heatmap, display_images

model = yolov8_heatmap(
    weight="/location/model.pt",
    conf_threshold=0.4,
    method="EigenCAM",
    layer=[10, 12, 14, 16, 18, -3],
    ratio=0.02,
    show_box=True,
    renormalize=False
)

imagelist = model("/location/image.jpg")
display_images(imagelist)
```

Codefragment 5.2: Code voor het genereren van heatmaps met YOLOv8_Explainer

5.3.3. Gekozen methode: EigenCAM

Voor dit onderzoek werd gekozen voor de methode EigenCAM vanwege de volgende voordelen: het werkt onafhankelijk van het objectlabel (geen class discriminatie), levert vaak heldere en interpreteerbare resultaten, en is geschikt voor het begrijpen van activatiezones zonder de ruis van class-based bias.
In tegenstelling tot GradCAM of LayerCAM, die sterk focussen op class-gecentreerde activatie, biedt EigenCAM een objectief beeld van visuele focuspunten in het model, gebaseerd op de eerste hoofdcomponent van de activatiekaarten.

5.3.4. Toepassing binnen dit project

YOLOv8_Explainer werd gebruikt op meerdere beelden uit zowel de training als testset. De gegenereerde heatmaps werden vergeleken met de oorspronkelijke annotaties om te beoordelen of het model correct focust op schadezones.
De voorbeelden van gegenereerde heatmaps worden in latere secties besproken, waarbij zowel correcte detecties (met activatie op de juiste plaats) als foutdetecties (met activatie op irrelevante delen zoals voeten of schaduwpartijen) worden geanalyseerd.
Deze visuele ondersteuning bleek nuttig om zowel correcte als foutieve beslissingen beter te begrijpen en worden geïntegreerd in de eindanalyse van het gekozen model.

5.3.5. Referentie

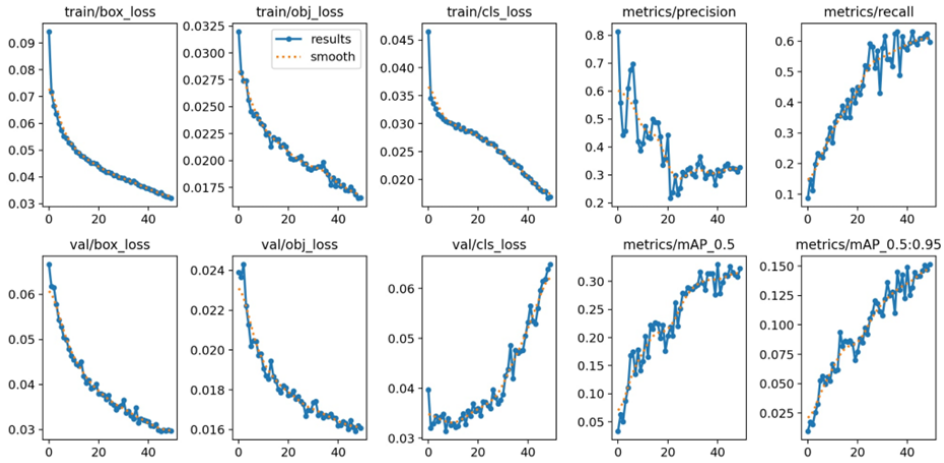
De implementatie maakt gebruik van EigenCAM-techniek via het YOLOv8_Explainer pakket. Dit pakket is gebaseerd op academische bronnen zoals Grad-CAM (Selvaraju et al., 2017), Grad-CAM++ (Chattopadhyay et al., 2018), en EigenCAM (Bany Muhammad & Yeasin, 2020), en werd in 2024 geciteerd in Journal of Physics: (Sarma Borah e.a., 2024).

5.4. Training van YOLOv5 op ongebalanceerde dataset

Om een eerste benchmark te verkrijgen van detectieprestaties op dakschadebeelden werd een YOLOv5s-model getraind op de oorspronkelijke ongebalanceerde dataset. De dataset bevatte acht verschillende klassen van schade met een sterke variatie in frequentie. Deze setup reflecteert een realistische beginsituatie waarin het datamateriaal nog niet geoptimaliseerd is voor machine learning.

5.4.1. Trainingsverloop en verliescurves

De training werd uitgevoerd over 50 epochs zonder gebruik van balancingtechnieken of aangepaste samplingstrategieën. Figuur 5.1 toont het verloop van de verschillende verlies- en evaluatiemetrieken tijdens de training.



Figuur 5.1: Trainingscurves YOLOv5 over 50 epochs op ongebalanceerde dataset

De onderste rij toont de validatieverliezen en evaluatiemetrieken. De belangrijkste observaties zijn:

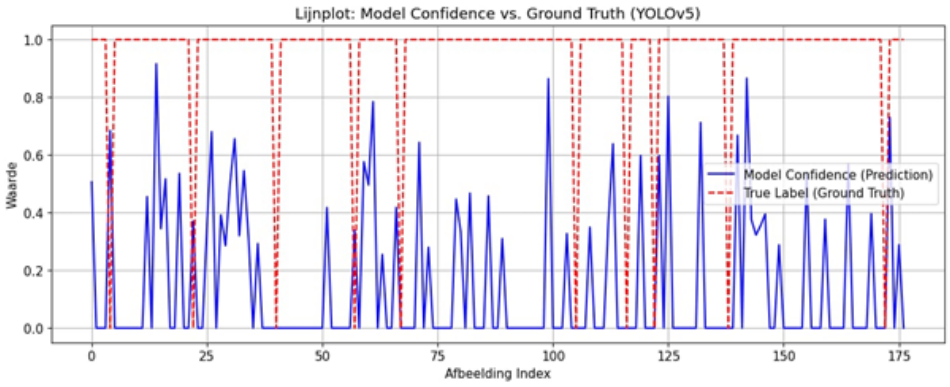
- Zowel val/box_loss als val/obj_loss dalen consistent, wat een positieve indicatie is voor generalisatie
- val/cls_loss begint na epoch 30 opnieuw te stijgen, een teken van beginnende overfitting
- De metrics/precision-curve piekt vroeg en daalt daarna, wat suggereert dat

het model aanvankelijk goed scoorde op bekende patronen, maar later generalisatieproblemen vertoont

- metrics/recall stijgt traag maar gestaag tot ongeveer 0.6, wat wijst op een groeiende maar nog beperkte capaciteit om relevante objecten op te sporen
- De mAP@0.5 en mAP@0.5:0.95 tonen een redelijk stijgende lijn, maar de absolute waarden blijven beperkt (rond 0.3 en 0.15 respectievelijk), wat typerend is voor een model dat last heeft van scheve dataverdeling

5.4.2. Testresultaten op externe dataset

Om de robuustheid van het getrainde model te toetsen werd een aparte testset gebruikt met beelden die niet voorkwamen in de originele training. Deze set bevatte extra variatie in belichting, perspectief en beeldresolutie, bijvoorbeeld foto's met voeten in beeld, extreme zoom en schaduwpartijen. Dit maakte het een uitdagende evaluatie voor elk model dat niet specifiek op dergelijke variatie is getraind. De resultaten van deze test zijn weergegeven in Figuur 5.2. De blauwe lijn vertegenwoordigt de voorspelde model confidence, terwijl de rode stippellijn de aanwezigheid van echte schadegevallen (ground truth) aanduidt.



Figuur 5.2: Confidence vs. Ground Truth voor YOLOv5

Enkele duidelijke observaties:

- Er zijn veel punten waar de ground truth op 1 staat, maar het model nauwelijks confidence toont (gemiste detecties)
- Sommige peaks in de confidence zijn fout-positieven, waarbij het model schade ziet waar er geen is
- De spreiding wijst op inconsistent gedrag, mogelijk door overfitting op dominante patronen in de trainingsdata

De classificatiematrix en prestatie-indicatoren bevestigen dit:

Confusion Matrix	[[7 3], [116 51]]
Precision	0.94
Recall	0.31
Accuracy	0.33

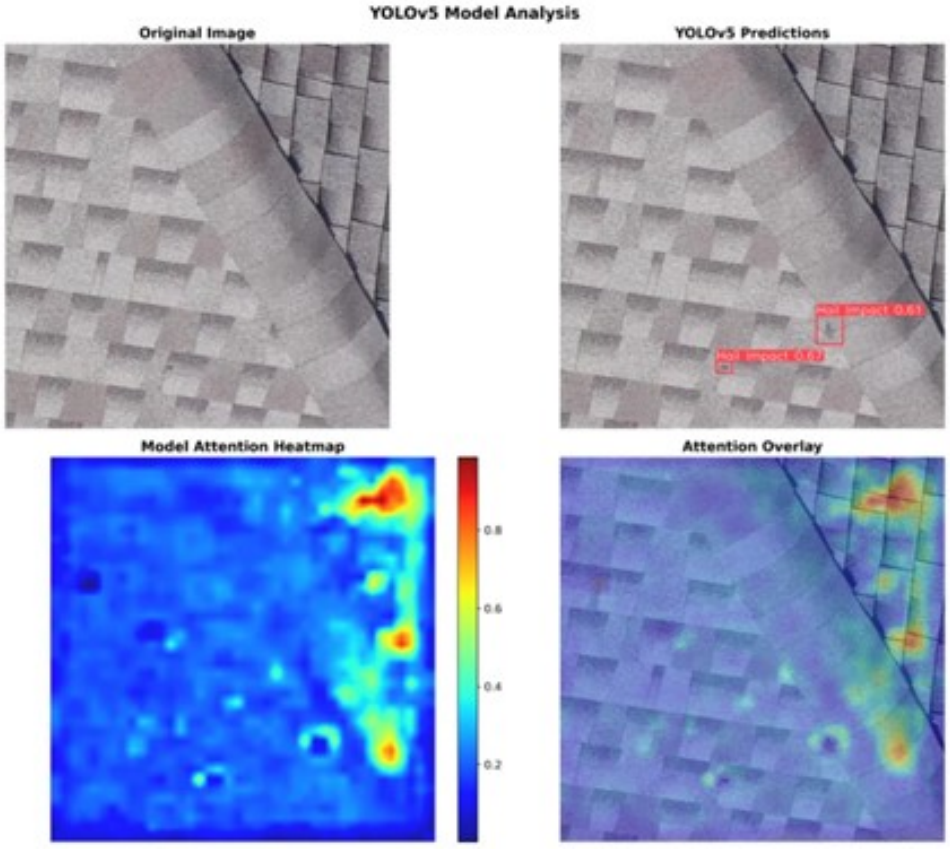
Tabel 5.1: Prestatie-indicatoren YOLOv5 op externe dataset

Hoewel de precision relatief hoog is (weinig vals positieven), is de recall zeer laag (veel schade wordt niet herkend). De totale accuracy ligt op 33%, wat onvoldoende is voor praktische toepassing.

5.4.3. Heatmapanalyse

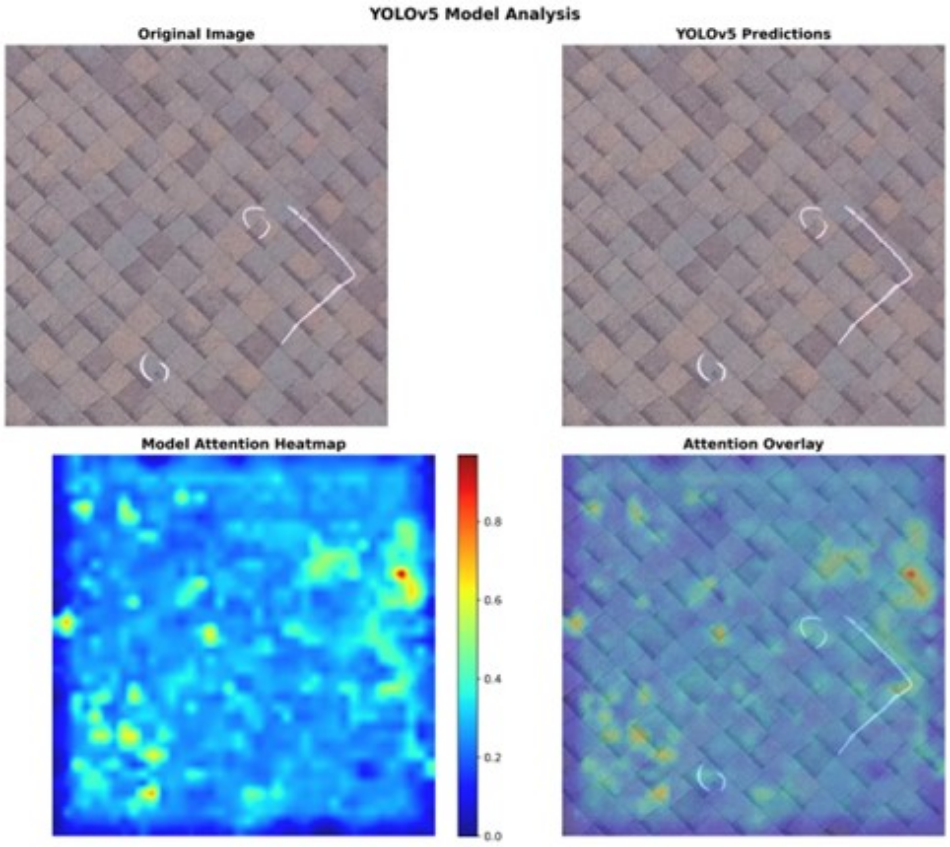
Voor een beter begrip van het modelgedrag werden heatmaps gegenereerd met behulp van EigenCAM-techniek. Deze analyse toont zowel succesvolle als mislukte detecties.

Figuur 5.3 toont een succesvolle detectie waarbij YOLOv5 correct "Hail Impact"iden-tificeert met confidence scores van 0.67 en 0.61. De bijhorende heatmap laat zien dat het model zich focust op de relevante schadezones, met warme (oranje/rode) activatie precies op de gedetecteerde locaties.



Figuur 5.3: YOLOv5 succesvolle detectie met correcte focus op schadezones

Figuur 5.4 illustreert daarentegen een gemiste detectie. Hoewel er visuele schade zichtbaar is, maakt het model geen voorspellingen. De heatmap toont verspreide activatie over het beeld, waarbij het model onder andere reageert op de witte krijtmarkeringen die door inspecteurs op het dak zijn aangebracht. Dit demonstreert hoe het model kan worden afgeleid door irrelevante visuele elementen.



Figuur 5.4: YOLOv5 gemiste detectie met verspreide activatie op irrelevante elementen

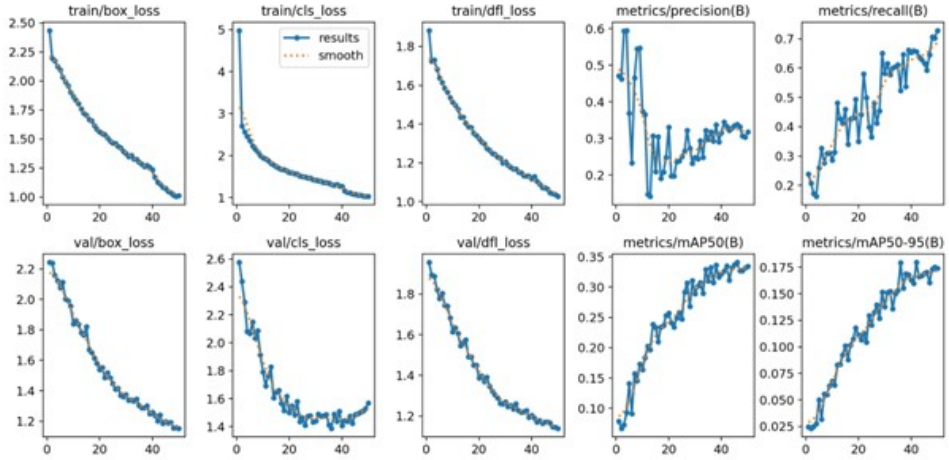
Deze uitkomst illustreert een klassiek probleem bij ongebalanceerde datasets: het model leert dominante klassen goed herkennen, maar faalt bij minder vaak voorkomende of visueel afwijkende schadegevallen.
Deze analyse bevestigt de bevindingen uit de kwantitatieve evaluatie: het model heeft moeite met consistente detectie en kan worden beïnvloed door visuele stoorfactoren in de omgeving.

5.5. Training van YOLOv8 op ongebalanceerde dataset

Na de eerste resultaten met YOLOv5 werd het trainingsproces herhaald met YOLOv8. Dit recentere model staat bekend om zijn verbeterde backbone-architectuur en betere integratie met moderne tooling. De training werd uitgevoerd over 50 epochs op dezelfde ongebalanceerde dataset.

5.5.1. Trainingscurves

De trainingsresultaten van YOLOv8 zijn weergegeven in Figuur 5.5. De evolutie van de loss-functies (box_loss, cls_loss en dfl_loss) en prestatiemetrieken (precision, recall, mAP@0.5 en mAP@0.5:0.95) wordt getoond voor zowel training als validatie.



Figuur 5.5: YOLOv8 trainingsgrafieken over 50 epochs op ongebalanceerde dataset

Analyse van de resultaten:

- De loss-curves dalen gestaag, wat een gezond leerproces weergeeft. De train/dfl_loss, specifiek voor YOLOv8, vertoont een consistente afname
- De val/cls_loss begint na ongeveer 30 epochs te stijgen, wat kan wijzen op overfitting aan de ongebalanceerde trainingsdata
- metrics/recall(B) stijgt trager dan bij YOLOv5, maar toont uiteindelijk een hogere eindwaarde (>0.7), wat wijst op een beter detectievermogen
- metrics/precision(B) is iets instabieler, wat zou kunnen wijzen op meer foutpositieven

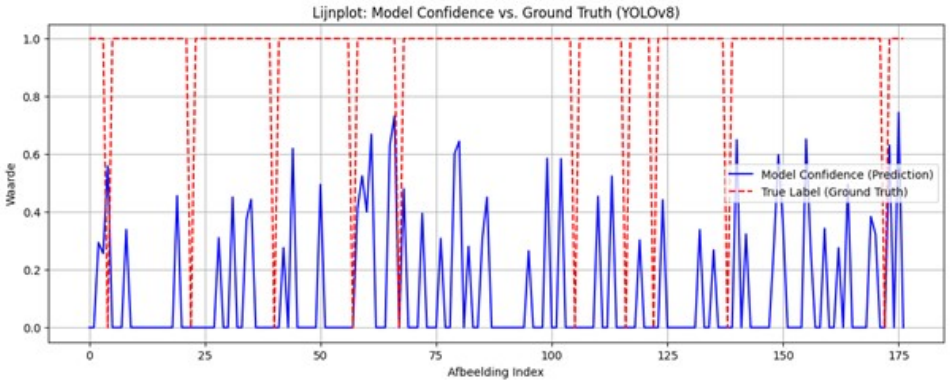
5.5.2. Test op Externe Dataset

Voor een eerlijke vergelijking werd YOLOv8 geëvalueerd op dezelfde externe dataset als YOLOv5. De resultaten van deze test zijn:

Confusion Matrix	[[9 1], [119 48]]
Precision	0.98
Recall	0.29
Accuracy	0.32

Tabel 5.2: Prestatie-indicatoren YOLOv8 op externe dataset

Hoewel de precision hoog blijft (minder fout-positieven), blijft de recall laag, wat betekent dat het model nog steeds veel schadegevallen mist. Net zoals bij YOLOv5 heeft het model moeite om zeldzamere schadeklassen betrouwbaar te herkennen, vooral in nieuwe omstandigheden.
De confidence-plot (Figuur 5.6) bevestigt deze bevindingen: de confidence blijft laag voor een groot aantal echte positieven en is vaak inconsistent, vergelijkbaar met de YOLOv5-resultaten.



Figuur 5.6: Model Confidence vs. Ground Truth (YOLOv8)

5.5.3. Heatmapanalyse

YOLOv8_Explainer werd gebruikt om visuele activatiekaarten te genereren en het gedrag van YOLOv8 te analyseren. De EigenCAM-techniek toont waar het model zijn aandacht richt tijdens detectie.
Voorbeelden van modelgedrag:

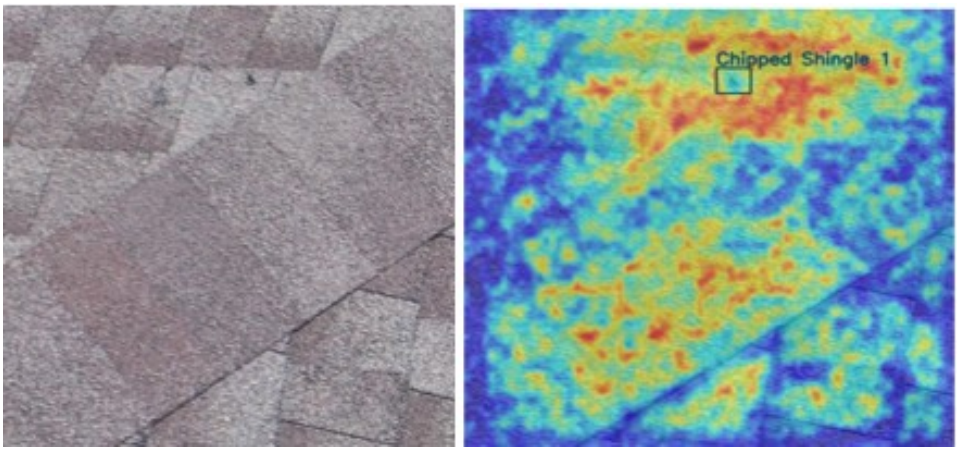
```
from YOLOv8_Explainer import yolov8_heatmap, display_images

model = yolov8_heatmap(
    weight="/location/model.pt",
    conf_threshold=0.4,
    method = "EigenCAM",
    layer=[10, 12, 14, 16, 18, -3],
    ratio=0.02,
    show_box=True,
    renormalize=False,
)

imagelist = model(img_path="/location/image.jpg")
display_images(imagelist)
```

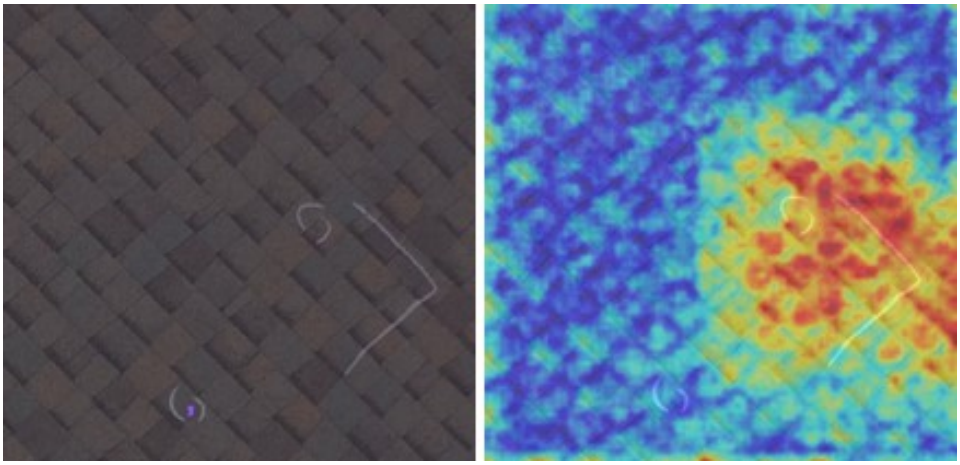
Codefragment 5.3: Code voor het genereren van YOLOv8 heatmaps

Succesvolle detectie: In het eerste voorbeeld is duidelijk te zien dat het model zijn aandacht concentreert op een zone die daadwerkelijk schade bevat, met intensiteit op de juiste plaats.



Figuur 5.7: YOLOv8 succesvolle detectie met correcte focus

Mislukte detectie: Het tweede voorbeeld toont dat het model in de buurt van de correcte bounding box focust, ook al is de detectie niet perfect. Dit is een significante verbetering tegenover het ongebalanceerde model uit sectie 5.4, waar de aandacht vaak verstoord werd door irrelevante visuele kenmerken zoals krijtmarkeringen.



Figuur 5.8: YOLOv8 gemiste detectie met verbeterde focus ten opzichte van vorige modellen

5.5.4. Reflectie op Prestaties

Ondanks de verbeterde architectuur van YOLOv8 en de iets hogere recall ten opzichte van YOLOv5, blijven de prestaties op de externe dataset beperkt.
Hoofdoorzaken van de beperkte prestaties:

- De originele dataset bevat relatief weinig variatie en is beperkt in omvang, waardoor het model gevoelig is voor overfitting

- De testset bevat veel visuele variatie (extreme zoomniveaus, schaduwen, externe markeringen) die weinig voorkwamen tijdens training
- De ongebalanceerde klassenverdeling zorgt ervoor dat het model onvoldoende wordt blootgesteld aan zeldzamere schadegevallen

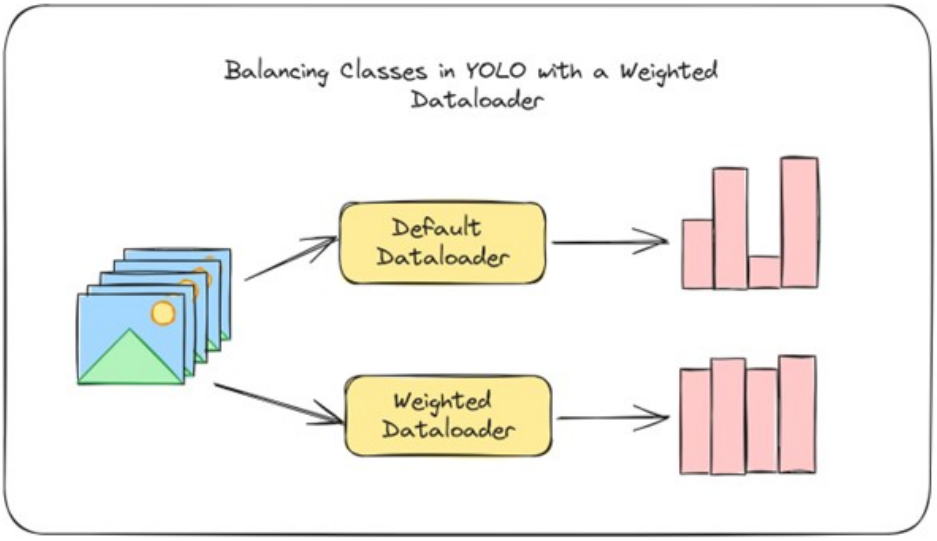
Deze bevindingen tonen de noodzaak aan van databalancing, wat in de volgende sectie wordt onderzocht.

5.6. Balancing met YOLOWeightedDataset

Data-onbalans vormt een van de grootste uitdagingen bij het trainen van object-detectiemodellen voor dakschade. De frequentieverschillen tussen klassen zoals 'damage' en zeldzamere types zoals 'Big-Damage' of 'Degranulation' leiden tot een model dat goed presteert op veelvoorkomende schade, maar faalt bij minder voorkomende categorieën. Om dit probleem aan te pakken werd een aangepaste oplossing geïmplementeerd: de YOLOWeightedDataset, gebaseerd op een gewogen dataloader (Yasin, 2024)
Voordelen van gewogen sampling:
De keuze voor een gewogen dataloader (in plaats van verliesgewichten of undersampling) biedt drie belangrijke voordelen:

- Eenvoud: De gewogen sampling is eenvoudiger te implementeren en vereist geen aanpassing van de lossfunctie
- Volledige databehoud: In tegenstelling tot undersampling worden geen voorbeelden genegeerd
- Constantere gradiënten: Minder abrupte veranderingen in verlieswaarden doordat het model regelmatiger wordt geconfronteerd met zeldzame klassen

Figuur 5.9 illustreert het verschil tussen een standaarddataloader en de gewogen versie, waarbij de gewogen sampling de klassenverdeling corrigeert naar een evenwichtiger geheel.



Figuur 5.9: Balancing Classes in YOLO with a Weighted Dataloader

5.6.1. Implementatie van YOLOWeightedDataset

Op basis van de officiële Ultralytics YOLODataset werd een aangepaste YOLOWeightedDataset geïmplementeerd. In deze implementatie krijgt elke afbeelding een samplingkans op basis van de relatieve zeldzaamheid van de klassen die ze bevat. De samplingkansen worden bepaald met de SQRT-strategie, die een gebalanceerd compromis biedt tussen agressieve en conservatieve aanpassingen van samplinggewichten.

```
class YOLOWeightedDataset(YOLODataset):
    def __init__(self, *args, balance_method="sqrt", debug=True,
                 **kwargs):
        super().__init__(*args, **kwargs)
        self.train_mode = "train" in self.prefix
        self.balance_method = balance_method
        self.debug = debug

        self.count_instances()
        self.class_weights = self.calculate_sqrt_weights()
        self.weights = self.calculate_weights()
        self.probabilities = self.calculate_probabilities()
```

Codefragment 5.4: YOLOWeightedDataset klasse initialisatie

Technische implementatie:

- count_instances(): Telt annotaties per klasse in de dataset

- calculate_sqrt_weights(): Berekent SQRT-gebaseerde gewichten voor balancing
- calculate_weights(): Bepaalt finale samplingkansen per afbeelding
- calculate_probabilities(): Converteert gewichten naar probabiliteiten

Sampling mechanisme: In plaats van sequentiële verwerking wordt tijdens training een gewogen willekeurige selectie uitgevoerd. Dit verhoogt de representatie van zeldzame klassen zonder de originele dataset te modificeren.

```
def __getitem__(self, index):
    if not self.train_mode:
        return self.transforms(self.get_image_and_label(index))
    else:
        weighted_index = np.random.choice(len(self.labels),
                                          p=self.probabilities)
        return
            self.transforms(self.get_image_and_label(weighted_index))
```

Codefragment 5.5: Gewogen sampling implementatie in YOLOWeightedDataset

De SQRT-methode biedt een evenwichtige aanpak tussen agressieve herbalancing en behoud van de originele dataverdeling.

Waarom SQRT-strategie?

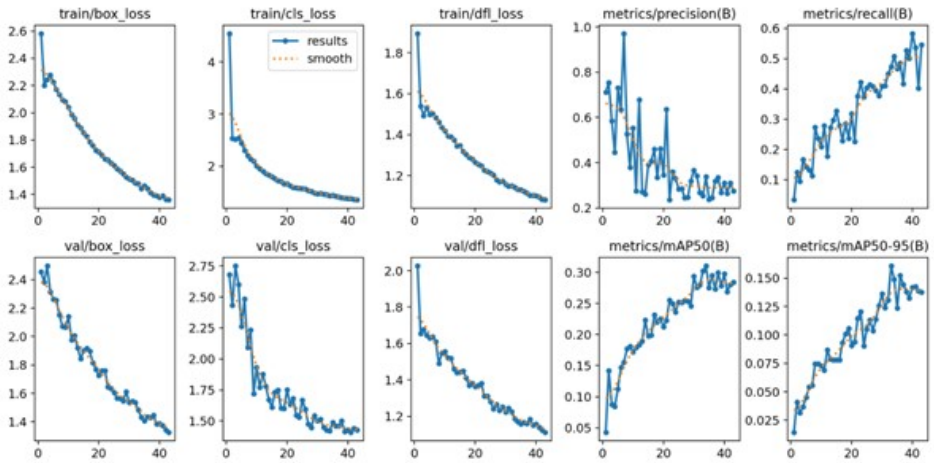
De keuze voor de SQRT-strategie is gebaseerd op praktische overwegingen uit eerdere experimenten. Tijdens verkennende tests bleken agressievere methoden zoals inverse weighting te leiden tot instabiele training, waarbij het model overfit raakte op de kleinste klassen. Conservatievere benaderingen daarentegen hadden onvoldoende effect op de klassenonbalans.
SQRT biedt een tussenweg: het geeft zeldzame klassen substantieel meer gewicht (bijvoorbeeld 2-3x voor de minst voorkomende categorieën), maar voorkomt extreme overcorrectie die de trainingstabiliteit kan verstoren. Deze benadering heeft zich in vergelijkbare computer vision-toepassingen bewezen als robuust en praktisch implementeerbaar.
Bovendien sluit SQRT aan bij de natuurlijke schaalverhoudingen in veel datasets, waar de frequentieverschillen tussen klassen vaak exponentieel zijn. Door de wortel te nemen van de inverse frequenties wordt deze exponentiële verdeling gedempt tot meer beheersbare proporties.

5.7. Training van YOLOv8 met gebalanceerde dataset

Na het ontwikkelen van de YOLOWeightedDataset werd YOLOv8 opnieuw getraind met SQRT-balancing. Deze strategie heeft als doel zeldzame klassen zoals ”Big Damageöf ”Chipped Shingle”vaker te tonen tijdens training, terwijl dominante klassen zoals ”damage”proportioneel minder worden getoond.
Door de implementatie van gewogen sampling kunnen we nu de trainingsblootstelling per klasse bewust sturen. Waar voorheen het model overwegend geconfronteerd werd met veelvoorkomende schadetypen, zorgt de SQRT-balancing voor een meer evenwichtige verdeling. Dit betekent dat zeldzame maar belangrijke schadepatronen nu voldoende aandacht krijgen tijdens het leerproces, zonder dat de meest voorkomende klassen volledig worden genegeerd.
Het effect hiervan is tweeledig: enerzijds wordt overfitting op dominante patronen verminderd, anderzijds krijgt het model de kans om robuuste representaties te leren voor alle typen dakschade die in de praktijk kunnen voorkomen.

5.7.1. Trainingsresultaten met SQRT-balancing

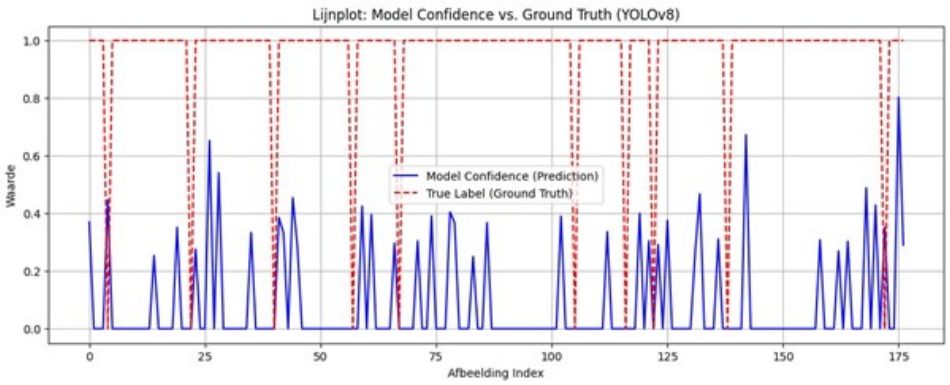
De trainingscurve in Figuur 5.10 toont een vlotte daling in de train/box_loss, train/cls_loss en train/dfl_loss, met eveneens dalende validatiewaarden. De mAP@0.5 en mAP@0.5:0.95 evolueren duidelijk positief, wat erop wijst dat het model beter generaliseert over alle klassen. In tegenstelling tot het eerdere ongebalanceerde model blijft de stijging in recall en mAP veel consistenter voor zeldzame categorieën.



Figuur 5.10: Train/val loss en metriek-evolutie met YOLOv8 + SQRT-balanced sampling

5.7.2. Resultaten op de externe testset

De externe testset werd gebruikt om het model te evalueren buiten de distributie van de trainingsdata. Zoals te zien in de confusion matrix (Figuur 5.11), leidt de class balancing tot een verminderde focus op de dominante klasse ”damage”, wat resulteert in een lagere recall (0.22) en accuracy (0.25) op deze testset.



Figuur 5.11: Confusion matrix voor YOLOv8 met SQRT-balancing op externe testset

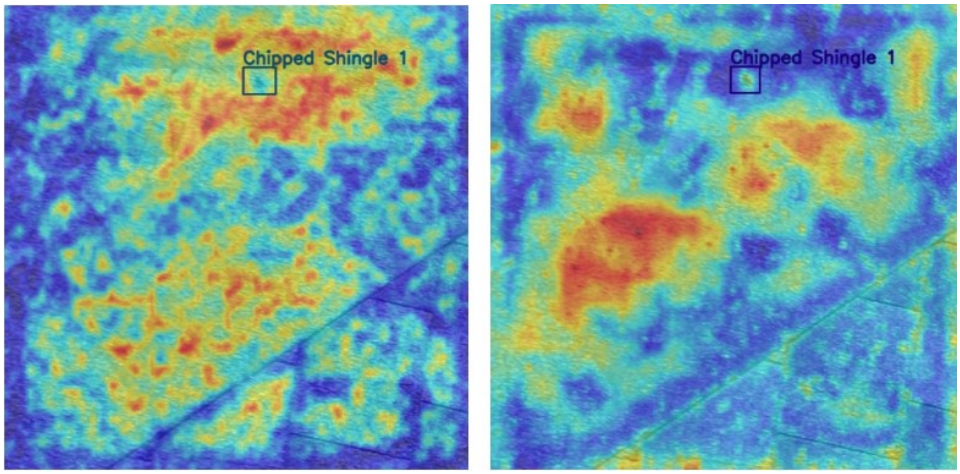
Hoewel dit op het eerste gezicht negatief lijkt, toont het aan dat het model minder overfit is op de dominante klasse. In plaats daarvan verdeelt het nu zijn aandacht beter over alle soorten schade, wat een gewenste eigenschap is in een realistische use-case met diverse schadebeelden.

5.7.3. Verbeterde visuele aandacht volgens heatmaps

De EigenCAM-heatmaps tonen aan dat het gebalanceerde model doelgerichter focust op relevante zones bij schade-identificatie. Deze verbetering is duidelijk zichtbaar in vergelijking met de eerdere ongebalanceerde modellen.

Succesvolle detectie met correcte focus

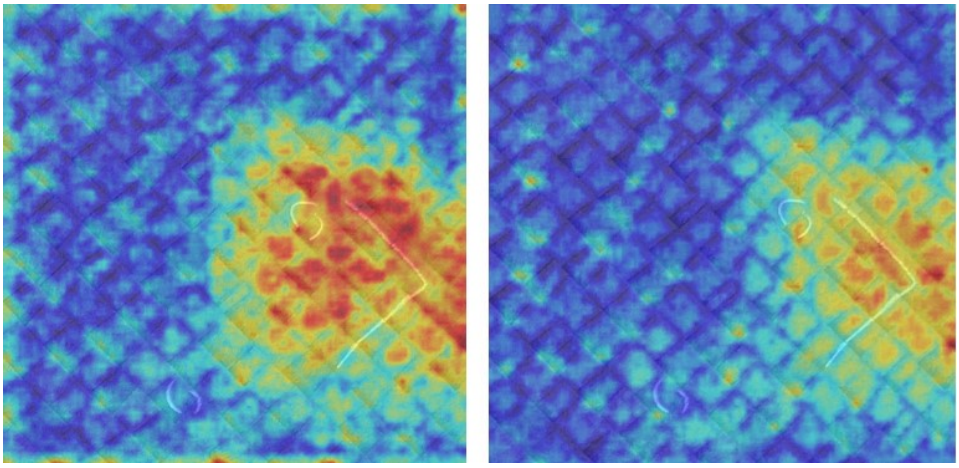
Figuur 5.12 toont een duidelijke hotspot op een correcte locatie, waar het model effectief ”Chipped Shingle”detecteert. De heatmap concentreert zich precies op het beschadigde gebied, wat aantoont dat het model de juiste visuele kenmerken heeft geleerd.



Figuur 5.12: Succesvolle detectie met correcte heatmap-focus na SQRT-balancing

Verbeterde focus bij gemiste detecties

In Figuur 5.13, waar geen voorspelling werd gemaakt, zien we toch een warme zone in het gebied waar visuele schade is. Dit suggereert dat het model de juiste zones leert herkennen, maar mogelijk onder de confidence threshold blijft. Dit is een significante verbetering ten opzichte van de ongebalanceerde training, waar de activatie vaak verstoord werd door irrelevante elementen zoals krijtmarkeringen.



Figuur 5.13: Verbeterde focus op relevante zones ondanks gemiste detectie

5.7.4. Conclusie SQRT-balancing

De toepassing van SQRT-balancing heeft geleid tot verschillende belangrijke verbeteringen:

Positieve effecten:

- Verhoogde aandacht voor zeldzame klassen tijdens training
- Betere generalisatie over het volledige klassenpalet
- Verbeterde visuele focus, zoals aangetoond in de heatmaps
- Verminderde overfitting op dominante klassen

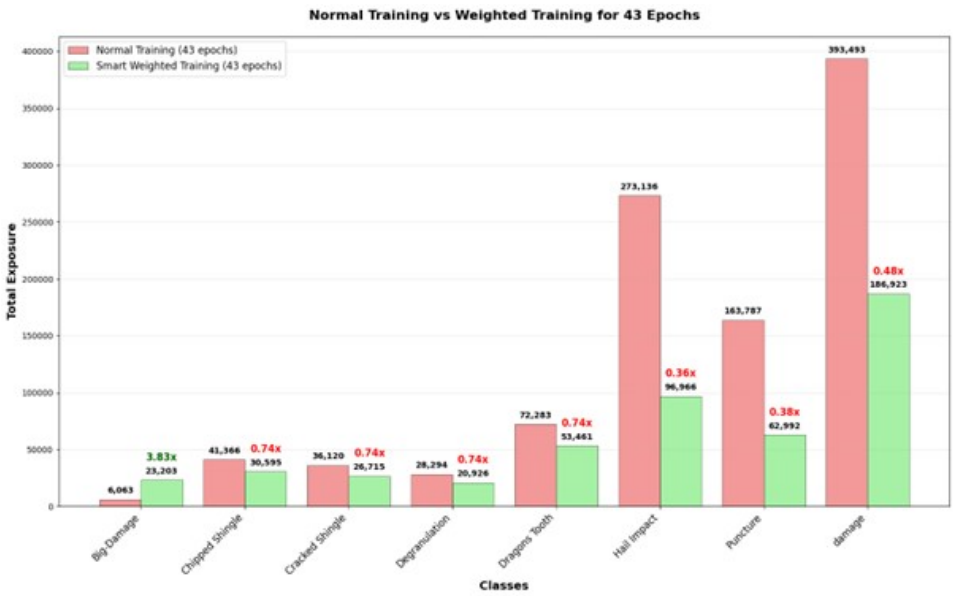
Trade-offs:

- Logische prestatievermindering op datasets met klasse-ongelijkheid
- Lagere recall voor de meest voorkomende klasse ”damage”
- Verminderde traditionele accuracy-metrics op ongebalanceerde testsets

Hoewel dit model op sommige testcases slechter scoort in traditionele metrics, vormt het inhoudelijk een robuustere en eerlijkere detectieoplossing. Dit is cruciaal in situaties waarin elk type schade even belangrijk is voor een volledige dakschade-evaluatie, onafhankelijk van de frequentie waarmee deze schadetypen voorkomen in de trainingsdata.

5.7.5. Visualisatie van blootstelling per klasse

Figuur 5.14 toont de totaal aantal keer dat elke klasse getoond werd tijdens training in 43 epochs. We vergelijken hier normale training (rood) met SQRT-balanced training (groen).



Figuur 5.14: Vergelijking van klasblootstelling tussen standaardtraining en SQRT-balanced training voor 43 epochs

- Minder voorkomende klassen zoals Big-Damage kregen een 3.83x hogere blootstelling bij gewogen training
- Dominante klassen zoals damage en hail impact werden daarentegen minder vaak getoond (0.48x en 0.36x)
- Andere klassen zoals Chipped Shingle, Cracked Shingle en Degranulation werden matig geherbalanceerd (0.74x)

Deze verdeling toont aan dat het model nu meer evenwichtige leerkansen kreeg over alle klassen. In plaats van zich te specialiseren in de dominante klassen, werd het model verplicht ook kennis op te bouwen over zeldzamere schadebeelden.

5.8. Vergelijkende evaluatie op gevarieerde dataset

Om de prestaties van de verschillende modelversies objectief te vergelijken, werd een additionele evaluatie uitgevoerd op een gevarieerde dataset. Deze dataset bevat een bredere selectie van dakschadebeelden met verschillende visuele condities en schadetypes.

5.8.1. Prestatiemetrieken per modelversie

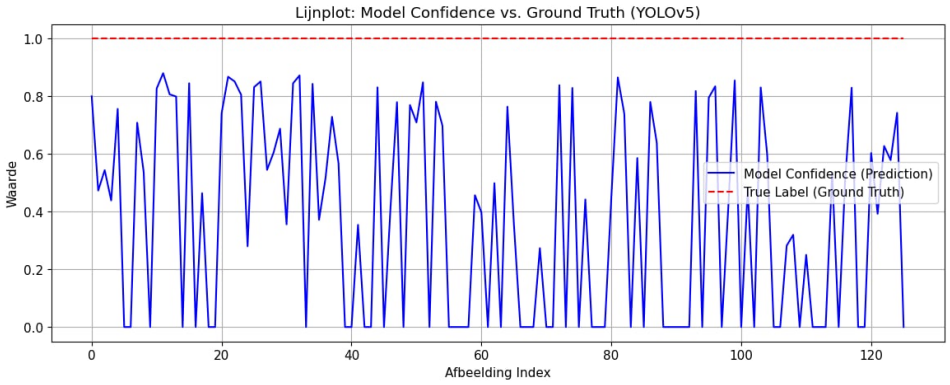
De resultaten van YOLOv5, YOLOv8 ongebalanceerd, en YOLOv8 gebalanceerd op de gevarieerde dataset zijn samengevat in Tabel 5.3.

Model	Confusion Matrix	Precision	Recall	Accuracy
YOLOv5	[[0, 0], [52, 74]]	1.00	0.59	0.59
YOLOv8 ongebalanceerd	[[0, 0], [43, 83]]	1.00	0.66	0.66
YOLOv8 gebalanceerd	[[0, 0], [29, 97]]	1.00	0.77	0.77

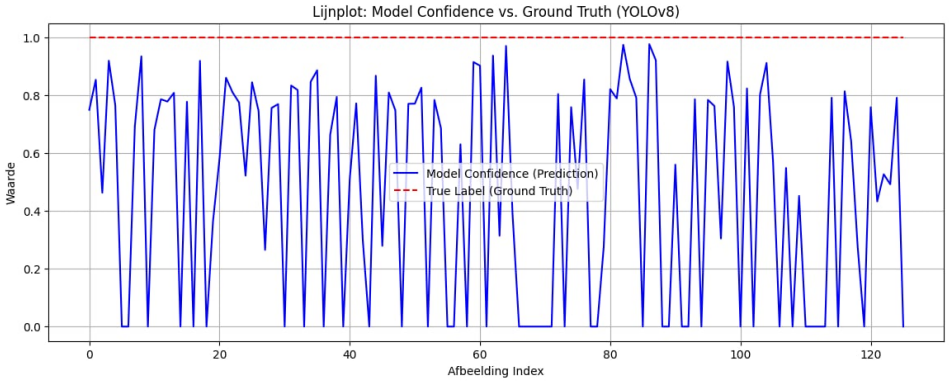
Tabel 5.3: Prestatiemetrieken van YOLOv5, YOLOv8 ongebalanceerd en YOLOv8 gebalanceerd op gevarieerde dataset

5.8.2. Analyse van confidence-patronen

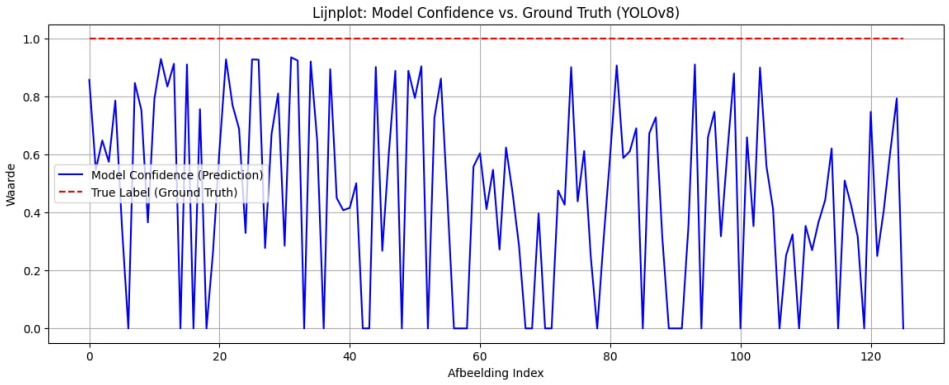
De confidence-plots in Figuren 5.15, 5.16, en 5.17 tonen een duidelijke evolutie in modelgedrag.



Figuur 5.15: Model Confidence vs. Ground Truth - YOLOv5 op gevarieerde dataset



Figuur 5.16: Model Confidence vs. Ground Truth - YOLOv8 ongebalanceerd op gevarieerde dataset



Figuur 5.17: Model Confidence vs. Ground Truth - YOLOv8 gebalanceerd op gevarieerde dataset

De resultaten tonen een duidelijke verbetering in recall van 0.59 (YOLOv5) naar 0.77 (YOLOv8 gebalanceerd), wat aantoont dat de combinatie van architectuurverbetering en databalancing effectief is voor betere schadedetectie.

5.8.3. Vergelijkende analyse en conclusie

De evolutie van YOLOv5 naar YOLOv8 gebalanceerd toont een duidelijke vooruitgang in dakschadedetectie, ondanks enkele trade-offs die inherent zijn aan de gekozen aanpak.

Belangrijkste bevindingen

De vergelijkende evaluatie toont drie essentiële inzichten:

Architectuurverbetering: De overgang van YOLOv5 naar YOLOv8 resulteerde in een recall-verbetering van 0.59 naar 0.66 op de gevarieerde dataset. Deze verbetering is toe te schrijven aan de geavanceerdere feature extraction en objectlocalisatie van de nieuwere architectuur.

Impact van databalancing: De implementatie van SQRT-balancing bracht de recall verder omhoog naar 0.77, wat een substantiële verbetering van 30

Visuele interpretabiliteit: De heatmap-analyses tonen dat het gebalanceerde model niet alleen beter presteert, maar ook begrijpelijker focust op relevante schadepatronen. Dit verhoogt het vertrouwen in de praktische toepasbaarheid.

Beperkingen en uitdagingen

Ondanks de vooruitgang blijven er uitdagingen bestaan die de absolute prestaties beïnvloeden:

Beperkte datasetomvang: De relatief kleine trainingsset maakt het model gevoelig voor overfitting, zelfs met databalancing. Dit verklaart waarom de prestaties op externe datasets nog steeds beperkt blijven.

Visuele variabiliteit: De testsets bevatten extreme variaties in zoomniveaus, belichtingsomstandigheden en externe markeringen die weinig voorkwamen in de trainingsdata. Deze domain gap blijft een uitdaging.

Klassenonbalans in evaluatie: Hoewel het gebalanceerde model beter generaliseert over alle klassen, scoort het lager op testsets die nog steeds gedomineerd worden door de klasse ”damage”. Dit is echter een gewenste eigenschap voor een robuust detectiesysteem.

Aanbevelingen voor toekomstige ontwikkeling

Voor verdere verbetering van de dakschadedetectie worden de volgende stappen aanbevolen:

Dataset-uitbreiding: Een substantiële vergroting van de trainingsset met meer diverse voorbeelden per klasse zou de generalisatiecapaciteit significant kunnen verbeteren. Specifiek zou focus moeten liggen op het verzamelen van meer voorbeelden van zeldzame schadetypen.

Data-augmentatie: Geavanceerde augmentatietechnieken kunnen de effectieve datasetgrootte verhogen en het model robuuster maken tegen verschillende visuele condities zoals variërende belichting en schaduwen.

Transfer learning: Het gebruik van pre-trained modellen die getraind zijn op verwante domeinen (bijvoorbeeld algemene bouwschade of dakstructuren) zou kunnen helpen bij het leren van robustere feature representations.

Ensemble-methoden: Het combineren van meerdere modellen, elk gespecialiseerd in verschillende aspecten van schadedetectie, zou de algehele prestaties kunnen verbeteren.

6

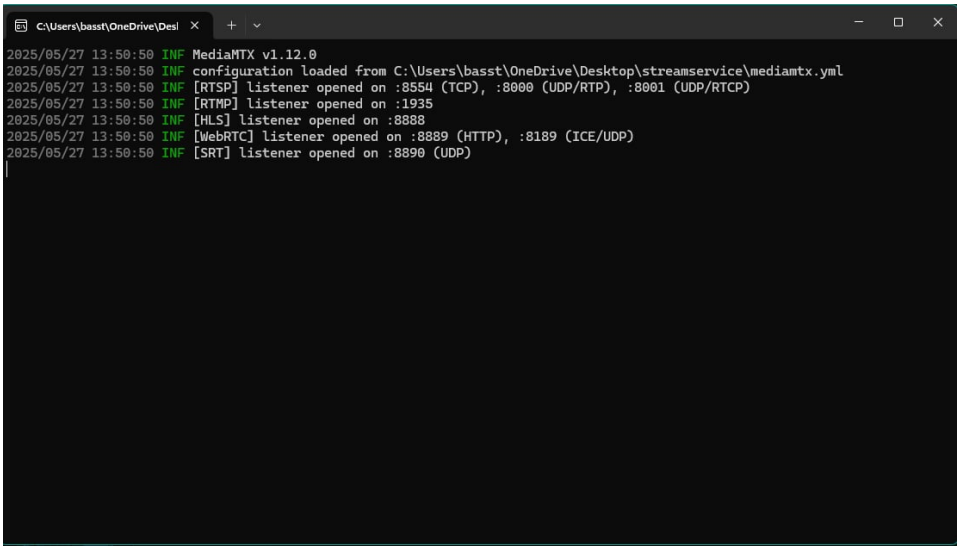
Real-time Streaming en Detectieproces

6.1. Doelstelling van het streaming- en detectieproces

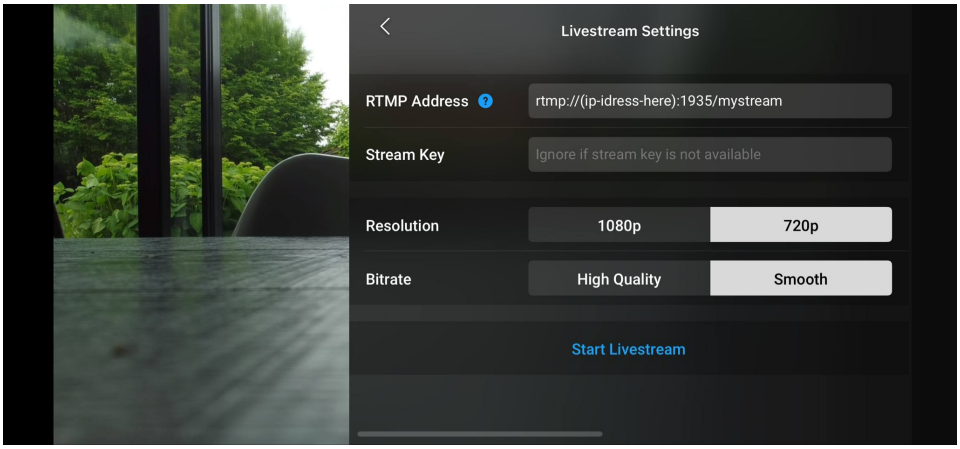
Dit hoofdstuk beschrijft de technische implementatie van het real-time detectiesysteem. Het systeem verwerkt camerabeelden van de drone terwijl deze in de lucht is, zodat schade onmiddellijk kan worden gedetecteerd. De beschrijving omvat de RTMP-streaming setup, de werking van het YOLO-detectiescript, en mogelijke uitbreidingen zoals patch-based inference.

6.2. Waarom RTMP en hoe de lokale server werd opgezet

Voor de video-overdracht tussen drone en laptop werd gekozen voor RTMP (Real-Time Messaging Protocol). Deze keuze werd gemaakt omdat RTMP stabiele overdracht biedt via TCP en goed integreert met OpenCV en PyTorch.
Een lokale RTMP-server werd opgezet met MediaMTX software. Figuur 6.1 toont de server in werking, waarbij verschillende protocollen actief zijn: RTSP op poort 8554, RTMP op poort 1935, en WebRTC op poort 8889. De configuratie werd geladen vanuit het bestand `mediamtx.yml`.
In de DJI-app werden de livestream-instellingen geconfigureerd zoals getoond in Figuur 6.2. Het RTMP-adres werd ingesteld op `rtmp://[ip-adres]:1935/mystream`, met 720p resolutie en 'Smooth' bitrate voor stabiele overdracht tijdens de vlucht.



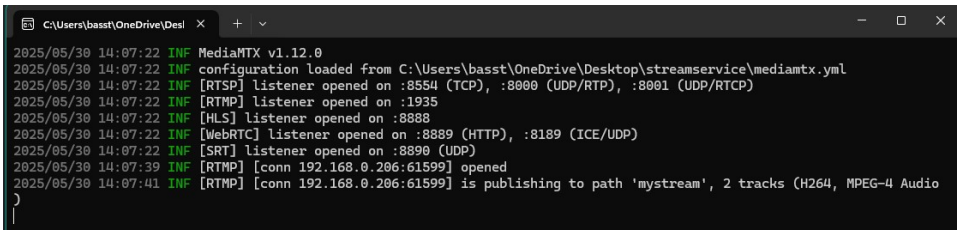
Figuur 6.1: MediaMTX server in werking met verschillende protocollen



Figuur 6.2: Livestream-configuratie in de DJI-app

6.3. Hoe het detectiescript werkt op een RTMP-stream

Zodra de MediaMTX server de stream ontvangt (zie Figuur 6.3), kan het detectie-script deze verwerken. Het script opent de RTMP-stream via OpenCV met `cv2.VideoCapture('rtmp` en verwerkt elk frame in real-time.



Figuur 6.3: MediaMTX server die de inkomende drone stream ontvangt

Na preprocessing (resize naar 640×640 pixels en normalisatie) voert YOLO inference

uit. Non-Maximum Suppression filtert overlappende detecties, waarna bounding boxes rond gedetecteerde schade worden getoond. Het volledige proces verloopt als volgt:

1. **Stream initialisatie:** Verbinding maken met de RTMP-stream
2. **Frame extractie:** Elk frame uit de videostream halen
3. **Preprocessing:** Afbeelding voorbereiden voor YOLO-model
4. **Inference:** Schadedetectie uitvoeren
5. **Post-processing:** Resultaten filteren en visualiseren
6. **Display:** Gedetecteerde schade tonen met bounding boxes

Deze cyclus wordt herhaald voor elk binnenkomend frame, waardoor continue real-time detectie mogelijk wordt.

6.4. Wat YOLO precies doet tijdens inference

Tijdens inference analyseert het getrainde YOLO-model elk frame op dakschade. Het model voorspelt per rastervak of er schade aanwezig is, welk type schade (damage, cracked shingle, etc.), en de locatie via bounding boxes. Voor dit project werd de confidence threshold ingesteld op 0.5 om false positives te beperken, terwijl het model specifiek getraind is op de dakschade-dataset uit hoofdstuk 4.

6.5. Patch-Based Inference als uitbreiding

Hoewel het systeem zoals hierboven beschreven goed werkt voor algemene detectie, kan het potentieel verder verbeterd worden met patch-based inference.

6.6. Aided Hyper Inference

Het SAHI (Slicing Aided Hyper Inference) framework biedt een generieke slicing aided inference en fine-tuning pipeline voor kleine objectdetectie (Akyon e.a., 2022). De techniek deelt inputbeelden op in overlappende patches, wat resulteert in relatief grotere pixelgebieden voor kleine objecten ten opzichte van de beelden die in het netwerk worden ingevoerd.
“Detection of small objects and objects far away in the scene is a major challenge in surveillance applications” (Akyon e.a., 2022). Dergelijke objecten bestaan uit een klein aantal pixels in het beeld en missen voldoende details, waardoor ze moeilijk te detecteren zijn met conventionele detectoren.

6.6.1. Technische implementatie

Bij slicing aided inference wordt het originele querybeeld opgedeeld in overlappende patches. Elke patch wordt geresized terwijl de aspectverhouding behouden blijft. Daarna wordt objectdetectie onafhankelijk toegepast op elke overlappende patch. Een optionele full-inference (FI) met het originele beeld kan worden toegepast om grotere objecten te detecteren. De overlappende predictieresultaten en eventuele FI-resultaten worden samengevoegd tot de originele grootte met behulp van Non-Maximum Suppression (NMS) (Akyon e.a., 2022).
Experimentele evaluaties met objectdetectie-baselines op de Visdrone en xView luchtfoto objectdetectie datasets tonen aan dat “the proposed inference method can increase object detection AP by 6.8%, 5.1% and 5.3% for FCOS, VFNet and TOOD detectors, respectively” (Akyon e.a., 2022).

6.6.2. Voordelen voor dakschade-detectie

Voor dakschade-detectie is slicing aided inference waardevol omdat “object detection tasks in high-resolution images generated by the high-end drone and surveillance cameras” vaak te maken hebben met kleine objecten die verloren gaan in conventionele detectie (Akyon e.a., 2022).
Het onderzoek toont dat modellen getraind op datasets zoals MS COCO “yield significantly lower accuracy on small object detection tasks in high-resolution images” (Akyon e.a., 2022). Dit probleem is relevant voor dakschade-detectie, waar kleine scheuren, hagelschade of ontbrekende dakpanfragmenten cruciaal zijn voor accurate inspectie.
De detectienauwkeurigheid kan verder worden verhoogd met slicing aided fine-tuning, wat resulteert in “a cumulative increase of 12.7%, 13.4% and 14.5% AP” voor verschillende detectoren (Akyon e.a., 2022).

6.6.3. Implementatiemogelijkheden voor het dakschade-detectiesysteem

Een hybride aanpak kan worden geïmplementeerd waarbij het huidige real-time systeem wordt uitgebreid met SAHI-functionaliteit voor verdachte gebieden. Wanneer het systeem een lage confidence score geeft of wanneer de operator handmatig een gebied markeert voor gedetailleerde analyse, kan slicing aided inference worden geactiveerd.
“The proposed technique is generic in the sense that it can be applied on top of any available object detector without any fine-tuning” (Akyon e.a., 2022). Dit betekent dat SAHI direct kan worden geïntegreerd met het huidige YOLOv8-gebaseerde detectiesysteem zonder architectuurwijzigingen.
Het framework verhoogt de computationele tijd lineair terwijl de geheugenvereisten gefixeerd blijven. “Computation and memory budgets can also be traded-off by adjusting the patch sizes, considering the target platform” (Akyon e.a., 2022). Dit maakt het geschikt voor implementatie binnen de huidige hardwarespecificaties

van het draagbare detectiesysteem.

6.6.4. Praktische toepassing

Voor dakwerkers betekent een dergelijke uitbreiding dat:

- Kleine hagelschade die niet zichtbaar is in 720p livestream wordt gedetecteerd
- Beginnende scheuren of subtiele kleurverschillen door vochtschade worden opgepikt
- Ontbrekende dakpanfragmenten of kleine beschadigingen aan dakgoten niet worden gemist

De implementatie kan functioneren via een “deep scan”-knop in de gebruikersinterface, waarbij een snapshot van de huidige cameraview wordt genomen en geanalyseerd met behulp van slicing aided inference voor maximale detectienauwkeurigheid.

6.7. Conclusie

Dit systeem toont aan dat dakschade-detectie mogelijk is tijdens drone-vluchten door gebruik te maken van YOLO en RTMP-streaming.
De RTMP-implementatie via MediaMTX zorgt voor een stabiele video-overdracht tussen drone en laptop. De 720p resolutie met ’Smooth’ bitrate zorgt voor betrouwbare streaming tijdens vluchten, waarbij TCP-gebaseerde overdracht zorgt voor consistente verbindingen. De integratie met OpenCV maakt real-time frame-processing mogelijk zonder complexe streaming-protocollen.
Het YOLO-detectieproces verwerkt elk frame efficiënt door preprocessing, inference en post-processing stappen. De confidence threshold van 0.5 balanceert detectiegevoeligheid met false positive reductie. Het systeem detecteert dakschade in real-time en toont resultaten onmiddellijk via bounding boxes.
Voor toekomstige verbeteringen biedt slicing aided inference meer mogelijkheden. Het SAHI-framework kan de detectie van kleine dakschade verbeteren met 6.8-14.5% hogere nauwkeurigheid, vooral relevant voor subtiele schade die verloren gaat in 720p streams.
Het beschreven systeem vormt een basis voor praktische dakschade-detectie, waarbij toekomstige uitbreidingen de detectiecapaciteiten verder kunnen verbeteren.

7

Praktijktest en Evaluatie

Dit hoofdstuk beschrijft de praktijktest van het ontwikkelde prototype onder realistische omstandigheden. Het bouwt verder op de technische implementatie uit het vorige hoofdstuk en evalueert de prestaties van het complete systeem. De resultaten en bevindingen uit deze test vormen de basis voor de conclusies en aanbevelingen in het volgende hoofdstuk.

7.1. Doel van de praktijktest

De praktijktest was een belangrijke fase om het prototype te testen onder echte omstandigheden. Het doel was niet om perfecte resultaten te krijgen, maar om problemen en beperkingen te vinden die nog verbeterd kunnen worden. Dit past bij een bachelorproef waar het leerproces net zo belangrijk is als de technische resultaten.
Het hoofddoel was om de volledige pipeline te testen - van drone streaming via RTMP naar real-time YOLO herkenning - onder realistische condities. Er werden bewust zowel successen als problemen gedocumenteerd om een eerlijk beeld te geven van hoe het prototype nu presteert.

7.2. Testopstelling en aanpak

De praktijktest werd uitgevoerd met de volledige technische setup zoals beschreven in hoofdstuk 3. Figuur 7.1 toont de gebruikte drone.
Technische configuratie:

- DJI drone geconfigureerd voor RTMP streaming naar `rtmp://[ip]:1935/live/mystream`
- MediaMTX server draaiend op de laptop
- Herkenningsscript actief met real-time frame processing



Figuur 7.1: Dji mini 2

- Live interface met herkenningsresultaten (zie Figuur 7.2)

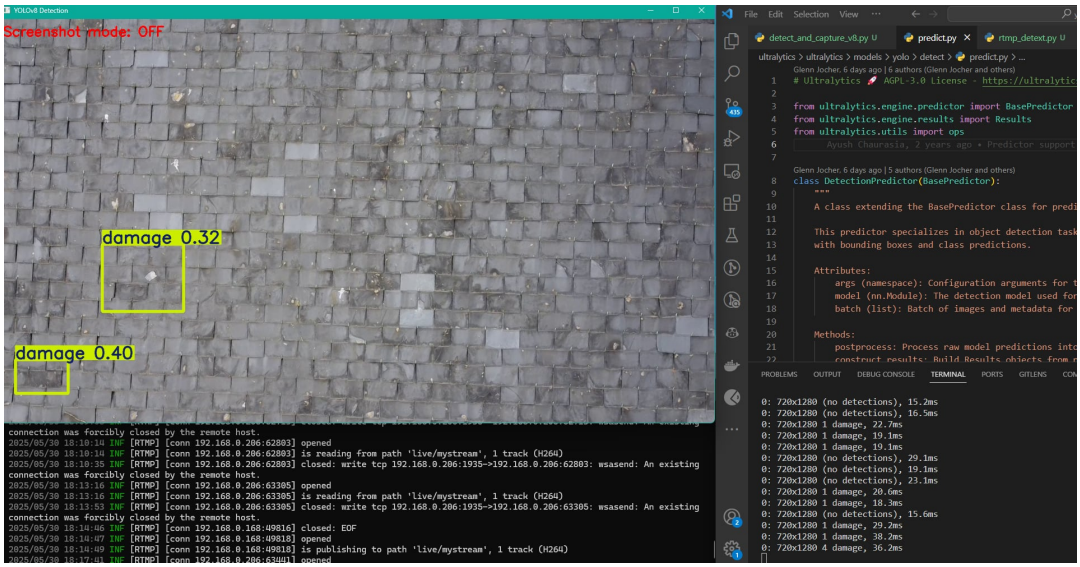
De test werd uitgevoerd op verschillende daktypen om te kijken hoe veelzijdig het systeem is. Er werd bewust gekozen voor zowel duidelijke als subtiele schadevormen om de grenzen van het systeem te testen.

7.3. Testresultaten en detecties

Figuur 7.5 toont detecties op grijze dakpannen met confidence scores tussen 0.35-0.47. Het systeem identificeerde verschillende schadegebieden met redelijke nauwkeurigheid.
Figuur 7.5 laat detecties zien op andere daktypes, waarbij confidence scores varieerden van 0.31 tot 0.64. Interessant was dat het systeem verschillende materialen en texturen aankon.

7.3.1. Prestatie metingen

Het systeem toonde de volgende prestaties tijdens de praktijktest. Tabel 7.1 geeft een overzicht van de gemeten RTMP stream latencies tijdens verschillende tijdstippen.
Tabel 7.2 vergelijkt de prestaties tussen CPU en GPU implementaties van het YOLO model.



Figuur 7.2: Live interface met real-time herkenningsresultaten tijdens de praktijktest



Figuur 7.3: dak variatie met confidence scores tussen 0.32-0.74

Tabel 7.1: RTMP Stream Latency Metingen

Tijd op klok	Tijd ontvangen	Latency (ms)
17:15:39	17:15:39.804	~804
17:16:15	17:16:15.185	~185
17:16:15	17:16:15.449	~449
17:17:11	17:17:11.714	~714

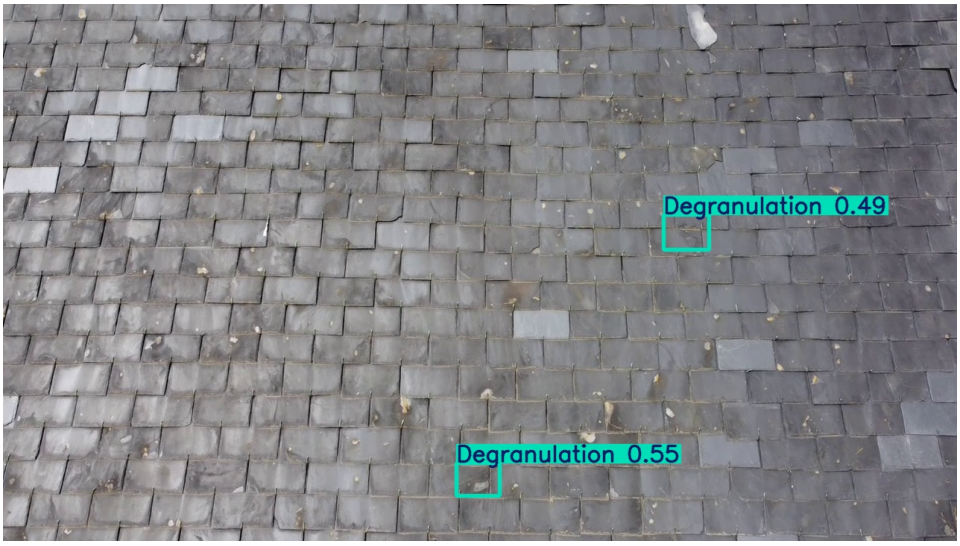
Tabel 7.2: YOLO Inference Prestaties Vergelijking

Metriek	CPU-versie	GPU-versie
Gemiddelde inference tijd	123.7 ms	~23.9 ms
Snelste frame	107.9 ms	10.8 ms
Traagste frame	156.3 ms	37.0 ms
Geschatte FPS	~8 fps	40-45 fps
Totale latency (RTMP + inference)	~600-700 ms	~520-540 ms

Interpretatie van de prestatiemetingen: Het systeem werkte goed in real-time, zowel op CPU als GPU. De GPU-versie toonde een dramatische verbetering met inference tijden die daalden van ~124ms naar ~24ms - een verbetering van ongeveer 80%. Bij GPU-inferentie werden zelfs 40-45 frames per seconde gehaald, wat ruim boven real-time ligt voor 384x640 input. De combinatie van RTMP latency (~500 ms) en GPU inference (~24 ms) gaf een totale vertraging van ongeveer 520-540 ms tussen wat er live gebeurt en wat gedetecteerd wordt.
De degranulation detecties gaven interessante resultaten met scores van 0.35-0.38, wat suggereert dat het model ook subtielere schade kan herkennen. De GPU-implementatie toonde niet alleen betere prestaties maar ook stabielere detecties door de verhoogde frame rate.



Figuur 7.4: Degranulation detectie 1 met subtiele schadepatronen



Figuur 7.5: Degranulation detectie 2 met subtiele schadepatronen

7.4. Problemen en beperkingen

7.4.1. Materiaalverschil als grote uitdaging

Een belangrijk probleem was het verschil tussen het trainingsmateriaal (vooral asphalt shingles) en het testmateriaal (dakpannen/leien). Dit verklaart waarom de confidence scores zo varieerden tussen 0.31 en 0.64 - het model was geoptimaliseerd voor een ander daktype dan waar er mee getest werd. Het systeem probeerde schadepatronen te herkennen die het geleerd had van shingles, maar moest deze toepassen op een materiaal met andere textuur, kleur en schadekenmerken.

7.4.2. Technische beperkingen

De 720p RTMP stream was een bottleneck voor detectie van zeer kleine schade. Kleine scheurtjes of impacts die wel zichtbaar zouden zijn in hoge resolutie foto's, verdwenen in de gecomprimeerde livestream. Dit werd versterkt doordat dakpannen vaak kleinere, meer lokale schadepatronen hebben dan de bredere schadegebieden die typisch zijn voor asphalt shingles.
De real-time verwerking introduceerde ook latency variaties die de detectiekwaliteit beïnvloedden. Wanneer de drone bewoog of de lichtcondities veranderden, had het systeem moeite om consistente detecties te houden. Dit resulteerde in detecties die soms wel en soms niet werkten in opeenvolgende frames.

7.4.3. Praktische problemen

Het vinden van de optimale vlieghoogte was complexer dan verwacht. Te hoog vliegen gaf onvoldoende detail voor schadedetectie, terwijl te laag vliegen leidde tot vlieginstabiliteit en motion blur in de video. Het dakpannenmateriaal vereiste een andere aanpak dan verwacht op basis van de shingle-training.

De weersafhankelijkheid beperkte de testmogelijkheden behoorlijk. Wind beïnvloedde niet alleen de vliegstabiliteit, maar ook de beeldkwaliteit door camera trillingen. Daarnaast zorgden veranderende lichtcondities voor variaties in de detectieprestaties, wat suggereert dat het model gevoelig is voor belichtingsverschillen tussen training en test omstandigheden.

7.5. Analyse en vervolgstappen

7.5.1. Positieve bevindingen

Het proof-of-concept toonde aan dat real-time dakschade detectie technisch mogelijk is met consumer hardware. Opvallend was dat het systeem, ondanks de training op asphalt shingles, toch schade kon detecteren op het geteste dakpannenmateriaal. Dit wijst op een zekere mate van flexibiliteit in het model. De GPU-implementatie bevestigde dat het systeem zeer goed schaalt met betere hardware - van 8 fps (CPU) naar 40-45 fps (GPU).

7.5.2. Belangrijke inzichten

De variatie in confidence scores (0.31-0.64) kwam vooral door het materiaalverschil tussen training en testdata. Dit laat zien hoe belangrijk het is om diverse trainingsdata te hebben die representatief zijn voor verschillende daktypen. De 720p streaming resolutie was een tweede knelpunt voor detectie van subtiele schade.

7.5.3. Aanbevelingen voor vervolgonderzoek

Voor een productierijp systeem zijn er drie hoofdverbeteringen nodig:

1. **Uitbreiding trainingsdata:** Diverse daktypen inclusief dakpannen en leien toevoegen aan de trainingsset
2. **Patch-based inference:** Implementatie voor betere detectie van kleine schadepatronen
3. **Streaming optimalisatie:** Hogere resolutie behouden zonder real-time prestaties te verliezen

7.6. Conclusie

Deze praktijktest laat zien dat het concept haalbaar is en identificeert concrete verbeterpunten. Het feit dat het systeem schade kon detecteren op een ander daktype dan waarvoor het getraind was, toont potentieel aan. De gevonden beperkingen geven duidelijke richtingen voor verdere ontwikkeling. De GPU-implementatie bewijst dat het systeem zeer goed schaalt en geschikt is voor real-time toepassingen.

8

Conclusie

Dit onderzoek bewijst dat real-time dakschade-detectie met drones technisch haalbaar is en identificeert concrete richtingen voor verdere ontwikkeling naar een praktisch systeem.

8.1. Beantwoording van de hoofdonderzoeksvraag

Het ontwikkelde systeem gebruikt YOLOv8 met SQRT-balancing en RTMP-streaming voor real-time detectie. Er wordt een real-time verwerking behaald met 40-45 FPS op GPU-hardware en toont aan dat de technische infrastructuur werkt. Resultaten varieerden afhankelijk van de testomgeving, waarbij het systeem beter presteerde op daktypen die overeenkwamen met de trainingsdata.

8.2. Prestaties en bevindingen

YOLOv8 presteerde consistent beter dan YOLOv5, wat de juiste keuze voor de modelarchitectuur bevestigt. De SQRT-balancing techniek verbeterde effectief de aandacht voor zeldzame schadetypen, wat een waardevolle bijdrage levert aan het onderzoeksveld van ongebalanceerde datasets.
Het verschil in prestaties tussen trainings- en testomgevingen benadrukt het belang van diverse datasets. Dit is een bekende uitdaging in computer vision die het belang van hiervan nogmaals benadrukt. De variatie in confidence scores (0.31-0.74) toont het potentieel voor optimalisatie door verfijnde modeltraining.

8.3. Technische doorbraak bewezen

Het onderzoek slaagt erin de volledige technische pipeline te demonstreren: van drone-opname via RTMP-streaming naar real-time AI-detectie op consumer hardware. De behaalde latentie van 520-540ms maakt praktische real-time feedback

mogelijk tijdens inspectievluchten. Dit is een opmerkelijke prestatie die de basis legt voor verdere ontwikkeling.

8.4. Relevantie en impact

Het onderzoek levert een werkend proof-of-concept dat zowel de mogelijkheden als de ontwikkelingsrichtingen voor real-time dakschade-detectie duidelijk maakt. Het prototype toont aan dat AI-detectie toegankelijk kan worden gemaakt voor kleinere bedrijven met betaalbare hardware.

Om in de praktijk toepasbaar te maken zijn er enkele verbeteringen nodig: datasetuitbreiding met diverse dakmaterialen, implementatie van patch-based inference voor kleine schadepatronen, en ontwikkeling van een gebruiksvriendelijke interface. Het ontwikkelde systeem biedt een solide basis om deze verbeteringen te implementeren en te testen.

9

Verdere uitbreiding

Het proof-of-concept toont aan dat real-time dakschade-detectie haalbaar is, maar biedt ook duidelijke richtingen voor verdere verbetering en uitbreiding. Dit hoofdstuk schetst concrete mogelijkheden om het systeem te optimaliseren voor praktische toepassing.

9.1. Datasetuitbreiding en diversificatie

De belangrijkste verbetering ligt in het uitbreiden van de trainingsdata. Het huidige model presteerde het best op daktypen die vergelijkbaar waren met de trainingsdata. Voor een betrouwbaar systeem is een dataset nodig die verschillende dakmaterialen omvat: dakpannen, leien, metalen daken en bitumineuze shingles. Ook verschillende weersomstandigheden, seizoenen en lichtcondities moeten worden opgenomen om de generaliseerbaarheid te verbeteren.
Een systematische dataverzameling in samenwerking met dakwerkersbedrijven en gemeenten kan voor deze diversiteit zorgen. Hierbij kunnen bestaande inspectierapporten worden gebruikt om gelabelde voorbeelden van echte schadegevallen te verzamelen.

9.2. Patch-based inference implementatie

De huidige 720p RTMP-stream vormt een bottleneck voor detectie van kleine schadepatronen. Implementatie van Slicing Aided Hyper Inference (SAHI) kan dit probleem aanpakken door hoogresolutie beelden op te delen in overlappende patches. Deze techniek kan de detectienauwkeurigheid verhogen met 6-14% volgens recente studies, wat vooral relevant is voor subtiele schade zoals beginnende scheuren of kleine hagelimpacts.
Een hybride aanpak waarbij het systeem verdachte gebieden markeert voor gedetailleerde patch-based analyse zou de beste balans bieden tussen snelheid en

nauwkeurigheid.

9.3. Streaming en hardware optimalisatie

Hoewel RTMP goed functioneert, kan onderzoek naar alternatieve streaming-protocollen zoals WebRTC of geoptimaliseerde GStreamer-pipelines de latentie verder verminderen. De huidige 520-540ms latentie is acceptabel, maar verbetering naar onder de 300ms zou de gebruikerservaring verder verbeteren.

9.4. Gebruikersinterface en workflow-integratie

Het huidige prototype focust op de detectietechnologie, maar praktische toepassing vereist een gebruiksvriendelijke interface. Een intuïtieve applicatie die inspecteurs begeleidt tijdens vluchten en automatisch rapporten genereert zou de overgang naar praktijkgebruik ondersteunen. Integratie met bestaande dakwerkersoftware en CRM-systemen zou de workflow verder optimaliseren.

9.5. Commerciële haalbaarheid en implementatie

Voor praktische implementatie moet het systeem worden getest in echte bedrijfsomgevingen. Pilotprojecten met kleine dakwerkersbedrijven kunnen feedback kunnen opleveren over gebruiksvriendelijkheid en beperkingen.
Een open-source ontwikkelingsmodel zou de adoptie kunnen versnellen en een community van ontwikkelaars en gebruikers kunnen opbouwen die bijdragen aan verdere verbetering.

9.6. Conclusie

De belangrijkste vervolgstappen zijn datasetuitbreiding, implementatie van patch-based inference en ontwikkeling van een gebruiksvriendelijke interface. Met deze verbeteringen kan het systeem evolueren van een proof-of-concept naar een werkbare oplossing die de veiligheid en efficiëntie van dakinspecties daadwerkelijk verhoogt.

A

Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

In de huidige bouwsector transformeert computer vision-technologie de manier waarop infrastructurele inspecties worden uitgevoerd. Traditionele visuele dakinspecties zijn vaak tijdrovend, kostbaar en brengen significante veiligheidsrisico's met zich mee voor inspecteurs. Volgens het Data Bulletin van februari 2021 van CPWR is de bouwsector een van de gevaarlijkste industrieën in de Verenigde Staten, met een toename van het aantal fatale ongevallen in 2019 (Brown e.a., 2021b).

A.1.1. Doelgroep en Probleemstelling

Hoewel drone-technologie de efficiëntie van inspectieprocessen heeft verbeterd, blijven commerciële oplossingen zoals DroneDeploy en Pix4D financieel moeilijk haalbaar voor kleine zelfstandige dakdekkersbedrijven. DroneDeploy's Professional Plan kost $3,499 per jaar (DroneDeploy, 2024), terwijl Pix4D's Professional licentie €4,990 per jaar kost (Pix4D, 2024). Volgens gegevens van Statbel (2022) heeft de meerderheid van de bedrijven in de bouwnijverheid (75%) slechts 1-4 werknemers. Dit duidt op een sector die sterk gedomineerd wordt door kleine ondernemingen (Statbel, 2022).

A.1.2. Centrale Onderzoeksvraag

Hoe kunnen open-source technologieën en betaalbare drone-hardware worden ingezet voor effectieve dakschade-detectie bij kleine dakwerkersbedrijven?

A.1.3. Deelvragen

Probleemdomein:

- Wat zijn de meest voorkomende uitdagingen bij traditionele dakinspecties?
- Wat zijn de financiële en operationele beperkingen van kleine dakwerkersbedrijven bij het implementeren van geavanceerde technologie?
- Welke soorten dakschade zijn het meest relevant voor kleine dakwerkersbedrijven?

Oplossingsdomein:

- Welke open-source detectiemodellen presteren het beste in de context van dakschade-detectie?
- Welke aanpak is het meest geschikt voor real-time detectie: verwerking op de drone zelf, post-flight analyse, of live-stream verwerking op een externe computer?
- Hoe kan een efficiënte video-streaming pipeline worden opgezet tussen drone en computer voor real-time verwerking?
- Welke datasets zijn beschikbaar voor het trainen van dakschade-detectiemodellen, en in hoeverre moeten deze worden aangevuld met eigen data?
- Welke minimale hardware-specificaties zijn nodig voor effectieve real-time verwerking op een laptop/PC?
- Hoe kan de latentie tussen video-capture en detectie worden geminimaliseerd?

A.1.4. Keuze voor Implementatie-aanpak

Na analyse van verschillende implementatiemogelijkheden richt dit onderzoek zich specifiek op real-time detectie via videostreaming naar een externe computer. Deze keuze is gemaakt omdat:

- Het directe feedback mogelijk maakt tijdens inspecties
- De verwerkingskracht van moderne laptops optimaal benut wordt
- De drone-hardware niet wordt belast met zware berekeningen
- Het een praktisch compromis biedt tussen real-time verwerking en verwerkingscapaciteit

A.1.5. Onderzoeksdoelstelling

De primaire doelstelling van dit onderzoek is het ontwikkelen van een betaalbare oplossing voor dakschade-inspectie die gebruikmaakt van open-source objectdetectiemodellen en drone-hardware. Specifiek richt het onderzoek zich op:

- **Analyse van Open-Source Detectiemodellen:** Identificeren en evalueren van detectiemodellen om het meest geschikte model te selecteren voor dakschadedetectie.
- **Proof-of-Concept:** Ontwikkelen van een werkend prototype dat post-flight schadedetectie uitvoert.
- **Optimalisatiebasis:** Opzetten van een basis voor verdere optimalisaties van het model en de verwerkingspipeline.

A.2. Literatuurstudie

De inzet van drones voor gebouwinspectie heeft de afgelopen jaren een aanzienlijke ontwikkeling doorgemaakt. Waar traditionele inspectiemethoden vaak gepaard gaan met risico's, tijdrovend zijn en hoge kosten met zich meebrengen, bieden drones een veiliger en efficiënter alternatief. Dankzij drone-technologie kunnen professionals hoogwaardige beeldmateriaal verzamelen zonder direct gevaar te lopen. Dit wordt ondersteund door onderzoek zoals beschreven in (Nooralishahi e.a., 2021), waar de veelzijdigheid en voordelen van drones in industriële inspecties worden toegelicht.

A.2.1. Objectdetectie en Computer Vision

Het analyseren van beelden voor dakschadedetectie kan op verschillende manieren worden uitgevoerd met behulp van computer vision-technieken. Moderne objectdetectie-algoritmes zoals YOLO (You Only Look Once), EfficientDet, en DETR (DEtection TRansformer) zijn bijzonder geschikt voor het identificeren van schadepatronen in dronebeelden. In de literatuur wordt benadrukt dat voor real-time verwerking de balans tussen snelheid en accuraatheid cruciaal is (of Engineering & (IRJET), 2019). Dit vereist zorgvuldige optimalisatie van de detectiemodellen voor gebruik op drone-hardware.

A.2.2. Open-Source versus Commerciële Oplossingen

Commerciële software zoals DroneDeploy en Pix4D wordt vaak geprezen om zijn uitgebreide functionaliteiten, waaronder geautomatiseerde kaartproductie en datavisualisatie. Echter, recent onderzoek toont aan dat open-source alternatieven steeds competitiever worden. Een studie van (Chen e.a., 2020) benadrukt dat bibliotheken zoals TensorFlow en OpenCV effectief kunnen worden geconfigureerd voor post-flight analyses, met vergelijkbare prestaties als commerciële tools, mits

ze correct zijn ingesteld. Deze bevindingen zijn belangrijk voor kleinschalige bedrijven die geen toegang hebben tot dure commerciële oplossingen.

A.2.3. Machine Learning en Schadedetectie

De effectiviteit van machine learning-algoritmes in schadedetectie blijft snel evolueren. Traditionele convolutionele neurale netwerken (CNN's) worden nu aangevuld met transformer-gebaseerde modellen en hybride benaderingen, zoals beschreven in (Yibo Sun e.a., 2024). Deze nieuwe modellen zijn in staat om complexere schadepatronen te herkennen, waaronder scheuren, lekkages en materiaaldefecten. Daarnaast tonen deze studies aan dat het mogelijk is om modellen specifiek te trainen op unieke schadepatronen die relevant zijn voor specifieke types daken of materialen.

A.2.4. Objectdetectie in Verwante Domeinen

Naast dakschadedetectie wordt objectdetectie ook onderzocht in andere domeinen, zoals de inspectie van bruggen. In (Alireza G. Kashani, 2015) wordt onderzocht hoe computer vision-methoden, zoals convolutionele neurale netwerken (CNN's), kunnen worden ingezet voor het identificeren van structurele schade aan bruggen. Hoewel de focus ligt op bruggen, zijn de onderliggende uitdagingen vergelijkbaar met die bij dakschadedetectie, zoals het detecteren van subtiele schadepatronen in complexe structuren. Dit suggereert dat de technieken die in bruginspectie worden onderzocht, mogelijk ook toepasbaar zijn voor dakschadedetectie.

A.2.5. Toepasbaarheid voor Kleinschalige Bedrijven

Hoewel er uitgebreide literatuur bestaat over de inzet van drones en geavanceerde computer vision-modellen voor industriële toepassingen, richt dit onderzoek zich specifiek op de bruikbaarheid van open-source technologieën voor kleinschalige dakdekkers. Hierbij ligt de nadruk op kostenbesparing, praktische toepasbaarheid en de mogelijkheid tot real-time verwerkingen, wat een essentiële behoefte is binnen deze sector. Deze niche blijft grotendeels onderbelicht in de huidige literatuur, waardoor dit onderzoek een waardevolle aanvulling vormt.

A.3. Methodologie

Dit onderzoek volgt een vergelijkende studie-aanpak, aangevuld met een proof-of-concept implementatie. De methodologie is opgedeeld in vijf fasen, waarbij elke fase concrete, meetbare resultaten oplevert. Voor het verzamelen van testmateriaal zal een DJI Mavic drone worden ingezet waarbij de camera-feed naar een computer wordt gestreamd voor real-time analyse van verschillende daktypes.

Fase 1: Vooronderzoek en Requirements Analyse (Duur: 3 weken)

Het onderzoek start met een grondige analyse van de technische vereisten en mogelijkheden:

- Uitvoeren van een diepgaande literatuurstudie naar real-time objectdetectiemodellen.
- Inventarisatie van beschikbare open-source tools en libraries geschikt voor real-time verwerking.
- Opstellen van concrete evaluatiecriteria met focus op verwerkingssnelheid en hardware-eisen.
- Analyse van de drone-hardware capaciteiten en beperkingen.
- Onderzoek naar verschillende streaming-mogelijkheden tussen drone en computer.

Deliverable: Gedetailleerd technisch rapport met evaluatiecriteria, geselecteerde tools en het aanbevolen detectiemodel voor real-time implementatie.

Fase 2: Data Verzameling en Preparatie (Duur: 4 weken)

Deze fase focust op het opbouwen van een representatieve dataset:

- Verzamelen van drone-testbeelden onder verschillende omstandigheden.
- Ontwikkelen van een systematische aanpak voor real-time video-analyse.
- Voorbereiden van trainingsdata voor het optimaliseren van real-time detectie.

Deliverable: Dataset met minimaal 1000 gelabelde voorbeelden van dakschade en testscenario's voor real-time detectie.

Fase 3: Implementatie en Ontwikkeling (Duur: 5 weken)

De kern van het onderzoek bestaat uit het implementeren van real-time detectie:

- Ontwikkeling van een Python-gebaseerde applicatie voor real-time videoverwerking.
- Implementatie van het geselecteerde model met optimalisaties voor snelle verwerking.
- Opzetten van een real-time evaluatieframework.
- Opzetten van een video-streaming pipeline.
- Ontwikkeling van een Python-gebaseerde applicatie voor het verwerken van de videostream.

Tools:

- Python 3.x als primaire programmeertaal
- OpenCV voor video-capture en beeldverwerking
- PyTorch of TensorFlow voor model implementatie
- Streaming tools voor video transmissie
- Git voor versiebeheer

Deliverable: Werkende real-time applicatie met geoptimaliseerd detectiemodel.

Fase 4: Evaluatie en Analyse (Duur: 3 weken)

Uitvoeren van systematische real-time tests:

- Kwantitatieve analyse van real-time detectie accuraatheid
- Latentie en framerate van de streaming pipeline
- CPU/GPU gebruik op de verwerkende computer
- Praktijktests onder verschillende omstandigheden

Deliverable: Uitgebreid evaluatierapport met real-time performantie metrics.

Fase 5: Validatie en Documentatie (Duur: 3 weken)

Afronden van het onderzoek:

- Validatie van real-time prestaties.
- Opstellen van technische documentatie.
- Schrijven van implementatierichtlijnen.
- Voorbereiden van de eindpresentatie.

Deliverable: Complete documentatie inclusief installatiehandleiding en demonstratie van real-time detectie.

A.4. Verwacht resultaat en conclusie

Dit onderzoek beoogt een significante bijdrage te leveren aan de toegankelijkheid van dakschade-detectie voor kleine dakwerkerbedrijven door middel van real-time detectietechnologie. De verwachte resultaten focussen zich op verschillende cruciale aspecten die de praktische toepasbaarheid van het systeem bepalen.

Technische prestaties

Voor de technische prestaties verwachten we de volgende meetbare resultaten:

- **Real-time Performance Metrics:**
  - Detectie-latentie: < 100ms per frame
  - Minimale framerate: 15 FPS
  - CPU gebruik: maximaal 70%
  - Stabiele videostream verwerking
- **Detectie Accuraatheid:**
  - Algemene detectie accuraatheid: > 85%
  - False positive rate: < 15%
  - False negative rate: < 10%

Deze prestatiemetrics zijn essentieel voor de praktische toepasbaarheid tijdens drone-inspecties. Het systeem zal worden geoptimaliseerd voor efficiënt gebruik van de beschikbare hardware, wat cruciaal is voor een soepele gebruikerservaring en betrouwbare detectieresultaten.

Praktische implementatie

De praktische implementatie zal resulteren in de volgende technische realisaties en operationele voordelen:

- **Technische Realisaties:**
  - Geoptimaliseerd model voor drone-hardware
  - Integratie met drone besturingssysteem
  - Gebruiksvriendelijke interface
  - Minimale setup-tijd voor operators
- **Operationele Voordelen:**
  - Betrouwbare werking onder verschillende lichtomstandigheden
  - Effectieve detectie op verschillende vlieghoogtes
  - Significante kostenbesparing ten opzichte van commerciële oplossingen
  - Basis voor verdere ontwikkeling

Uitdagingen en documentatie

We zijn ons bewust van mogelijke uitdagingen die de resultaten kunnen beïnvloeden:

- **Technische Uitdagingen:**
  - Hardwarematige beperkingen
  - Impact op batterijduur
  - Weersafhankelijkheid
  - Balans tussen nauwkeurigheid en snelheid

Alle resultaten zullen worden gedocumenteerd met gedetailleerde analyses, ondersteund door:

- **Documentatie:**
  - Performance grafieken
  - Confusion matrices
  - Praktische implementatierichtlijnen
  - Aanbevelingen voor optimalisatie

Het uiteindelijke doel is het creëren van een toegankelijke open-source oplossing die de balans vindt tussen kosten, prestaties en gebruiksgemak voor kleine dakwerkerbedrijven.

Bibliografie

Akyon, F. C., Altinuc, S. O. & Temizel, A. (2022). Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection. https://doi.org/https://doi.org/10.48550/arXiv.2202.06934

Alireza G. Kashani, A. J. (2015). Cluster-Based Roof Covering Damage Detection in Ground-Based Lidar Data. *Automation in Construction*. https://www.sciencedirect.com/science/article/abs/pii/S0926580515001508

Aloman, A., Ispas, A., Ciotirnae, P., Sanchez-Iborra, R. & Cano, M. (2015). Performance Evaluation of Video Streaming Using MPEG DASH, RTSP, and RTMP in Mobile Networks, 144–151. https://doi.org/https://doi.org/10.1109/WMNC.2015.12

Bochkovskiy, A., Wang, C. & Liao, H. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. *arXiv preprint*.

Brown, S., Harris, W., Brooks, R. D. & Dong, X. S. (2021a). *Fatal Injury Trends in the Construction Industry*. The Center for Construction Research en Training (CPWR).

Brown, S., Harris, W., Brooks, R. D. & Dong, X. S. (2021b). *Fatal Injury Trends in the Construction Industry*. The Center for Construction Research en Training (CPWR). Verkregen 15 november 2024, van https://www.cpwr.com/wp-content/uploads/DataBulletin-February-2021.pdf

Chen, X., Ye, Y., Zhang, X. & Yu, C. (2020). Bridge Damage Detection and Recognition Based on Deep Learning. *Journal of Physics: Conference Series*, *1626*(1). https://doi.org/10.1088/1742-6596/1626/1/012151

DroneDeploy. (2024). *DroneDeploy Pricing Plans*. Verkregen 21 maart 2024, van https://www.dronedeploy.com/pricing

Engineering, C. (2019). *7 Ways Drones Make Building Inspections Faster, Cheaper, And Safer*. https://web.archive.org/web/20200930100538/https://ctleng.com/7-ways-drones-make-building-inspections-faster-cheaper-safer/

Girshick, R., Donahue, J., Darrell, T. & Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. *Computer Vision and Pattern Recognition (CVPR), IEEE Conference*, 580–587.

Hezaveh, A. e.a. (2022). Automatic assessment of roofs using AI and UAVs. *Frontiers in Built Environment*.

Iriemon, K. (2020). *YOLOv4 - Neural networks for object detection (Windows and Linux version of Darknet)*. https://github.com/kiyoshiiriemon/yolov4_darknet

Laminar. (2022). *The Pros and Cons of Drone Building Inspections*. https://www.laminar.co/2022/08/10/the-pros-and-cons-of-drone-building-inspections/

LearnWithARobot. (2024). *YOLOv5 vs YOLOv6 vs YOLOv7: Speed and accuracy comparison*. https://www.learnwitharobot.com/p/yolov5-vs-yolov6-vs-yolov7

Markets & Markets. (2022). DRONE INSPECTION AND MONITORING MARKET GLOBAL FORECAST TO 2030. http://www.sbdi.co.kr/email/2021/20210802/sample/MnM_Drone_Inspection_Monitoring.pdf

Mekdad, Y., Aris, A., Babun, L., Fergougui, A. E., Conti, M., Lazzeretti, R. & Uluagac, A. S. (2023). A Survey on Security and Privacy Issues of UAVs. *Computer Networks*, *224*, 109626. https://doi.org/https://doi.org/10.1016/j.comnet.2023.109626

Nelson, J. & Solawetz, J. (2020). *YOLOv5 is here: State-of-the-art object detection at 140 FPS*. https://blog.roboflow.com/yolov5-is-here

Nooralishahi, P., Ibarra-Castanedo, C., Deane, S., López, F., Pant, S., Genest, M., Avdelidis, N. P. & Maldague, X. P. V. (2021). Drone-Based Non-Destructive Inspection of Industrial Sites: A Review and Case Studies. *Drones*. https://www.mdpi.com/2504-446X/5/4/106

of Engineering, I. R. J. & (IRJET), T. (2019). Applications of Object Detection System. *International Research Journal of Engineering and Technology*. https://www.irjet.net/archives/V6/i4/IRJET-V6I4920.pdf

OpenSistemas. (2024). *YOLOv9: A revolutionary breakthrough*. https://opensistemas.com

Pix4D. (2024). *Pix4D Pricing*. Verkregen 21 maart 2024, van https://www.pix4d.com/pricing

Redmon, J., Divvala, S., Girshick, R. & Farhadi, A. (2015). You Only Look Once: Unified, Real-Time Object Detection. *arXiv preprint*. https://doi.org/https://doi.org/10.48550/arXiv.1506.02640

Robotics, B. (2021). *GStreamer for video streaming (drones)*. https://blueye-robotics.github.io

Sarma Borah, P. P., Kashyap, D., Laskar, R. A. & Sarmah, A. J. (2024). A comprehensive study on Explainable AI using YOLO and post hoc method on medical diagnosis. *J. Phys. Conf. Ser.*, *2919*(1), 012045.

Singh, R., Kumar, A. & Sharma, P. (2024). Assessing the performance of YOLOv5, YOLOv6, and YOLOv7 in road defect detection and classification: A comparative study. *ResearchGate*. https://www.researchgate.net/publication/377880685

Statbel. (2022). *Demografie van de werkgevers in de profitsector*. Verkregen 21 maart 2024, van https://statbel.fgov.be/nl/themas/ondernemingen/demografie-van-ondernemingen/demografie-van-de-werkgevers-de-profitsector

Stereolabs. (2023). *YOLO v5 vs v7 vs v8 – Performance Benchmark*. https://www.stereolabs.com/en-be/blog/performance-of-yolo-v5-v7-and-v8

Team, R. (2024). 4 Reasons RTSP Streaming is Still Relevant. https://www.red5.net/blog/4-reasons-rtsp-streaming-is-still-relevant/

Ukhwah, E. N., Yuniarno, E. M. & Suprapto, Y. K. (2019). Asphalt pavement pothole detection using Deep Learning method based on Yolo Neural Network. *Proceedings of the 2019 International Seminar on Intelligent Technology and Its Applications (ISITIA)*. https://doi.org/10.1109/ISITIA.2019.8937176

Viso.ai. (2024). *Object Detection: Definitive Guide 2025*. https://viso.ai/deep-learning/object-detection/

Wang, C., Bochkovskiy, A. & Liao, H. (2023). A Comprehensive Review of YOLO Architectures in Computer Vision. *arXiv preprint*.

Yaseen, M. (2024). What is YOLOv9: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector. https://doi.org/https://doi.org/10.48550/arXiv.2409.07813

Yasin, M. (2024, september 1). *Balance Classes During YOLO Training Using a Weighted Dataloader*. https://y-t-g.github.io/tutorials/yolo-class-balancing/

Yibo Sun, Z. S. e.a. (2024). The Evolution of Object Detection Methods. *Automation in Construction*. https://www.sciencedirect.com/science/article/pii/S095219762400616X